Introduction to CUDA, Part I

Zuzanna Jedlinska

zuzanna@tacc.utexas.edu

https://github.com/zuzanna-m-j/IntroToCUDA_S2026.git

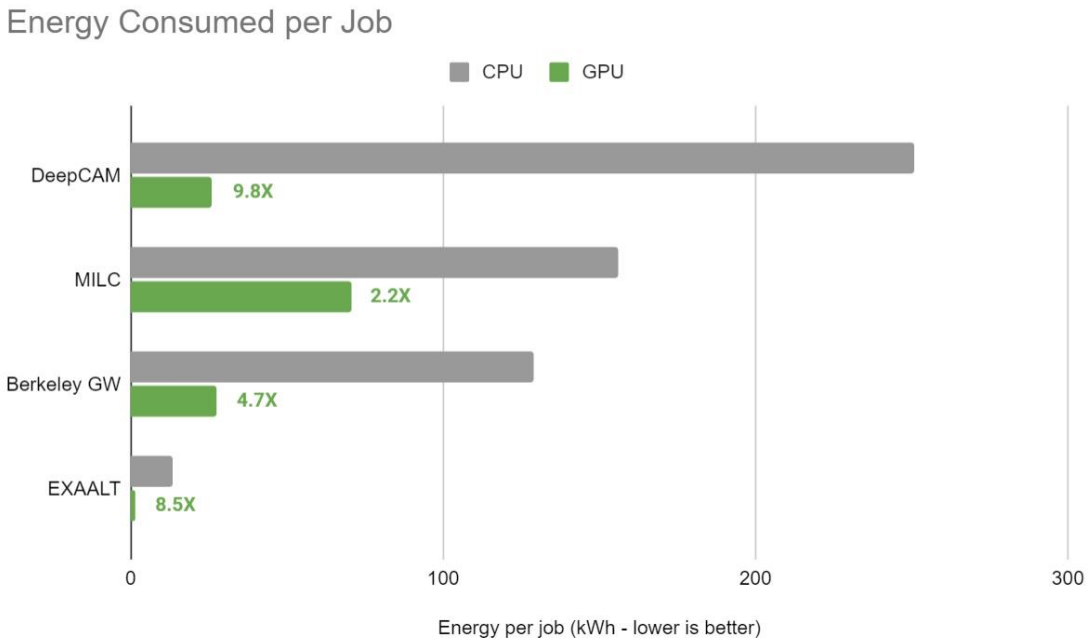
What is CUDA

- (Compute Unified Device Architecture)
- Parallel programming platform for NVIDIA GPUs
- Thread-based programming model
- Not a programming language but provides language extensions for C++, Fortran and Python
- Software developed to match the hardware

People: HTML is not a real programming language
CUDA:

Why use a GPU?

- Efficiency and performance
- CPU limiting factors:
  - (Mostly) Serial execution
  - Moore's law limit
  - Clock speed
- Benefits of using GPUs:
  - Massive parallelism
  - Energy efficiency
  - Python is now mainstream and everyone "knows it".
    But understanding CUDA still gives you the bragging rights.



https://blogs.nvidia.com/blog/gpu-energy-efficiency-nersc/

GPUs in scientific computing

- Machine Learning
- Molecular Dynamics
- Fluid Dynamics
- Quantum Mechanics
- Scientific Visualization

GPU is not a silver bullet

CPU programming is like using a handsaw to cut wood: straightforward, might be slower but eventually it will do its job. GPU programming is like a using a chain saw: it will cut wood faster if you know what you're doing, might cut your fingers faster if you don't.

Yes!

- Computation intensive tasks (high compute to data load ratio)
- Data accessed in large continuous chunks
- Matrix operations

No!

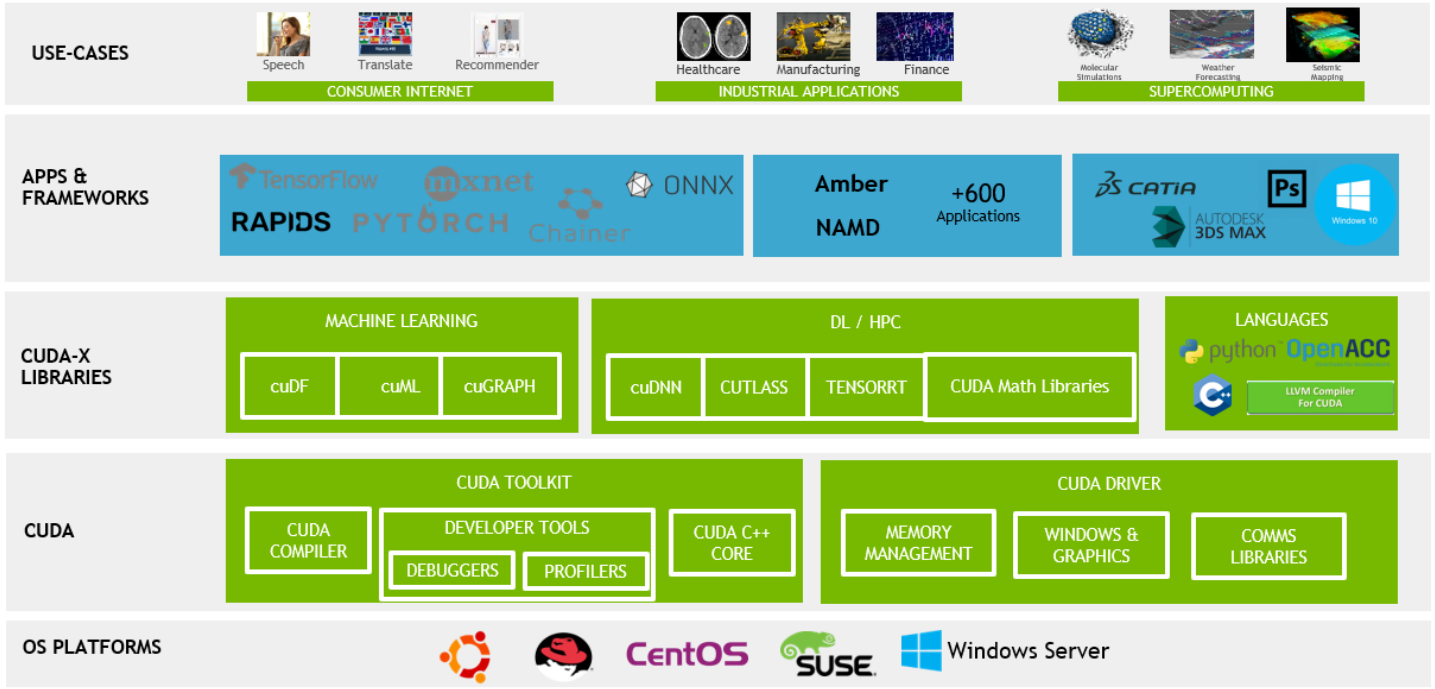
- Code with a lot of branching instructions
- Lots of interdependence between threads
- Processing done on small batches of data
- Irregular and scattered data access pattern

Ways to program a GPU

- GPU programming “languages”:
  - CUDA for NVIDIA (C/C++ and Fortran)
  - HIP for AMD (C only)
- Pragma (compiler directive) based device offloading:
  - OpenMP
  - OpenACC

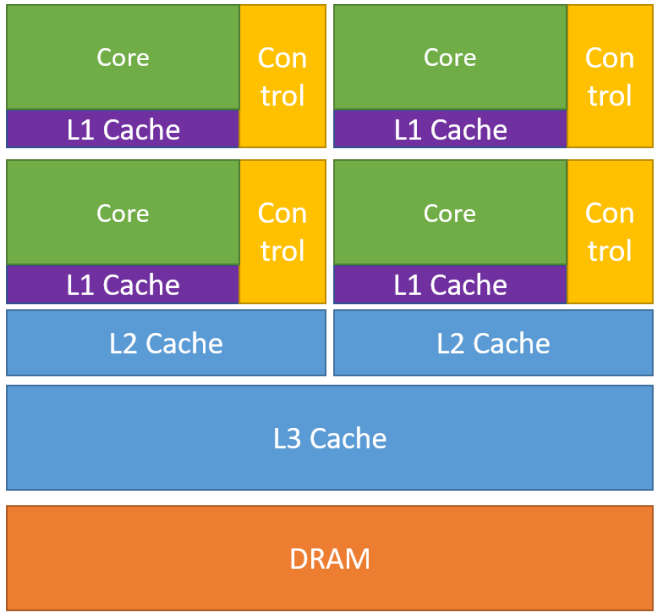
What's in the CUDA toolbox

- CUDA: C/C++ and Fortran
- Compilers: nvcc, nvc, nvc++, nvfortran
- Profiling tools: Nsight Systems, Nsight Compute
- Specialized libraries
  - Math: cuFFT, cuBLAS
  - Algorithms: Thrust
  - Deep Learning: cuDNN
- CUDA-aware MPI

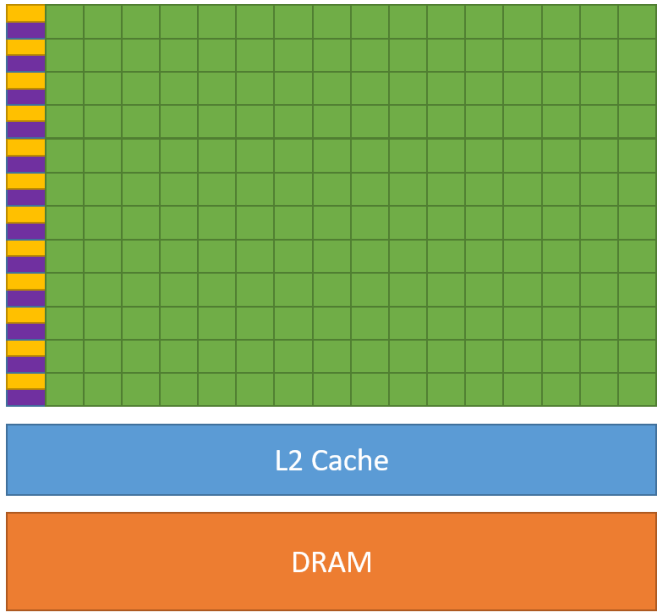


CPU vs GPU

CPU	GPU
Low core count	Very high (1,000s) core count
Low latency	High latency but high throughput
Good for serial execution, can be parallel	Optimized for parallel execution
Fast context switching, branching	Good for SIMD, avoid branching



CPU

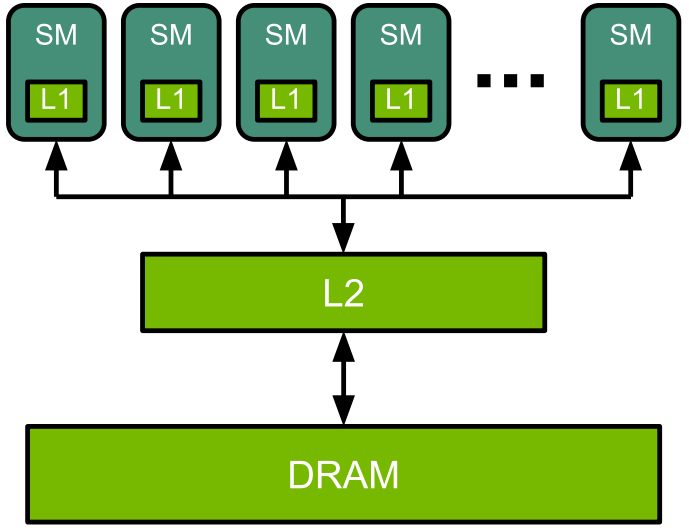


GPU

From NVIDIA's CUDA C++ Programming guide

GPU resource management
32 threads form a warp; warps organized in blocks; blocks organized into a grid
Kernel is called on a grid, resources only free up after kernel is done
Kernel launch always has overhead
Threads within a warp execute in step
No control over execution order of threads or blocks; concurrency within a warp

Multi-level of (NVIDIA) GPU organization



Physical Components
GPU card
Streaming Multiprocessors
Streaming Processors
CUDA cores / Tensor Cores

Memory
Global Memory (DRAM)
L2 Data Cache
L1 Data Caches
Registers

https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/
https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html#gpu-arch

Streaming Mutiprocessor (SM)

- Memory in SM:
  - L1 Data Cache / Shared Memory
- Divided into Streaming Processors (SP)
  - ➤ CUDA cores and Tensor core
  - ➤ Registers
  - ➤ Load/Store Units
  - ➤ Special Function Unit (cos(x), sin(x), e^x)

https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

What different numbers mean

- **Compute Capability (CC) -** defines what features and instructions are available for a specific GPU architecture.
- *See CUDA Programming guide 17.2. Features and Technical Specifications (Tables 27 & 28)*
  - Higher CC means more features.
  - Tesla (CC 1.x) → Hopper (9.0) → Blackwell 12.0
  - Example: Tensor cores available from CC 7.0+, thread block clusters 9.0+*CUDA*
- CUDA Toolkit (12.9.0 as of May 2025)

Feature	A100 80GB SXM	H100 80GB SXM
GPU Architecture	Ampere	Hopper
GPU Memory	80GB HBM2e	80GB HBM3
Memory Bandwidth	2039 GB/s	3352 GB/s
L2 Cache	40MB	50MB
FP64 Performance	9.7 TFLOPS	33.5 TFLOPS
FP32 Performance	19.5 TFLOPS	66.9 TFLOPS
RT Cores	N/A	N/A
TF32 Tensor Core	312 TFLOPS	989 TFLOPS
FP16/BF16 Tensor Core	624 TFLOPS	1979 TFLOPS
FP8 Tensor Core	N/A	3958 TFLOPS
INT8 Tensor Core	1248 TOPS	3958 TOPS

https://www.horizoniq.com/blog/h100-vs-a100-vs-l40s/

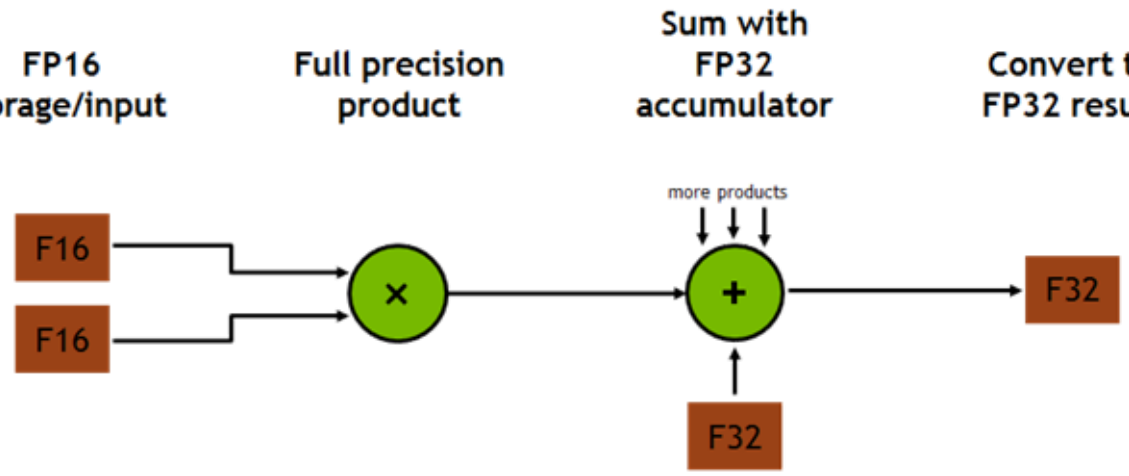
Tensor cores

- Specialized cores operate on 4x4 matrices
- Performed matrix multiply and accumulate
- Used

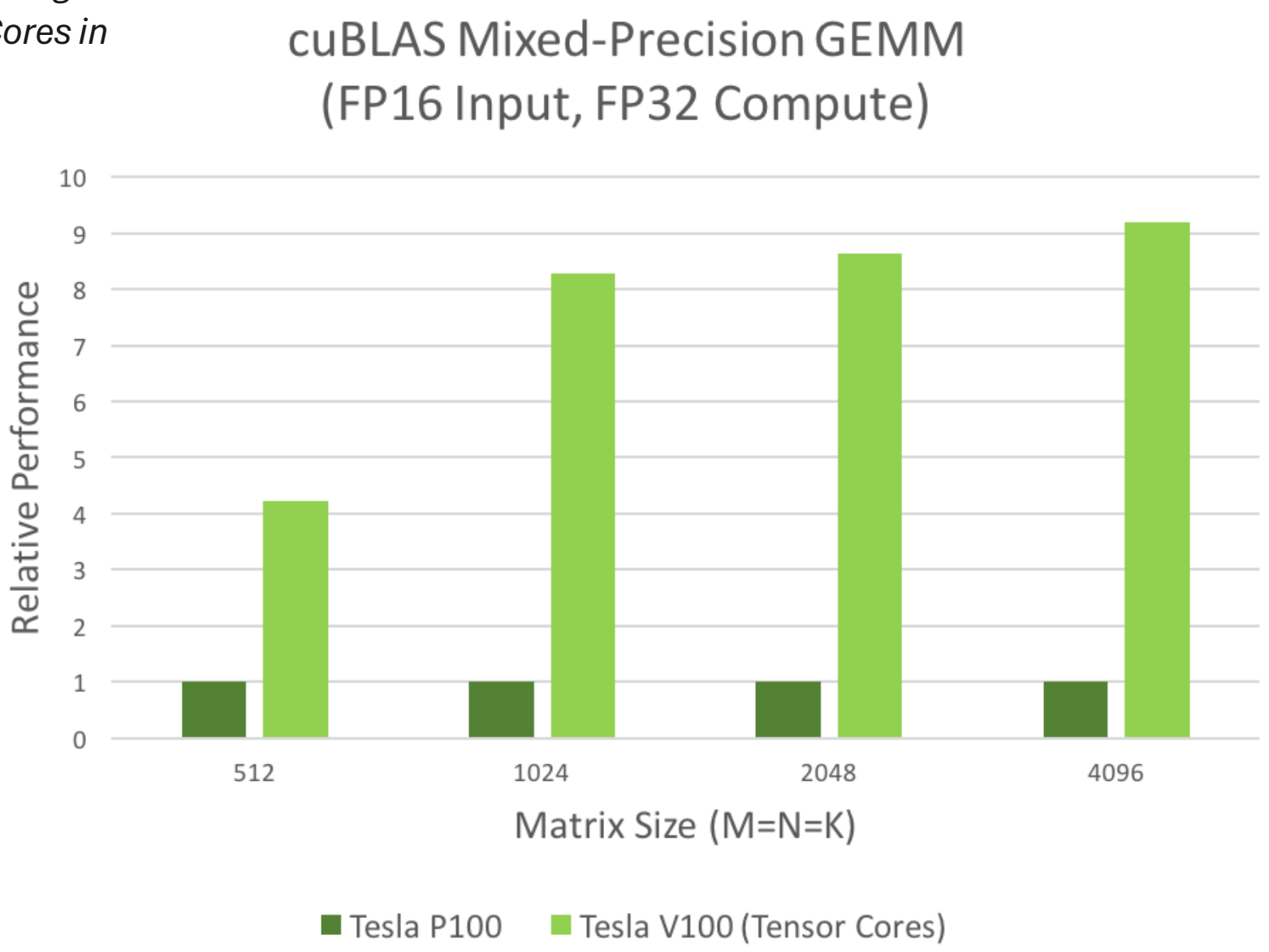
$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 FP16 or FP32

From Nvidia Technical Blog Programming Tensor Cores in CUDA 9



From Nvidia Technical Blog Programming Tensor Cores in CUDA 9



Useful resources

- **CUDA documentation** https://docs.nvidia.com/cuda/
- **NVIDIA Developer forums** https://forums.developer.nvidia.com/c/developer-tools/106
- **Stack Overflow** - its likely someone already had the same programming question as you and it has already been answered in the comments. *(Unlike reddit)* Stack Overflow has is mostly high quality, polite, informative answers.
- You can ask me, I will try my best to answer your question. You can also talk to your classmates, this way both of you learn.
- **Code:** https://github.com/zuzanna-m-j/IntroToCUDA_S2026.git

Login to Frontera and request a GPU node

```
ssh your_user_name@frontera.tacc.utexas.edu
idev –p rtx
module load cuda


git clone https://github.com/zuzanna-m-j/IntroToCUDA_S2026.git


cd IntroToCUDA_S2026
cd 0_Query
nvcc –o query query.cu
```

Query the device properties

- In the *0_Query directory, execute ./query*

```
c196-082[rtx](358)$ ./query

 * You have 4 CUDA-enabled devices *

 === Information for device number 0 ===
  Device name: Quadro RTX 5000
  Compute Capability: 7.5
  Maximum threads per SM:  1024
  Maximum threads per block (blockSize): 1024
  Max thread block size in x:1024, y:1024, z:64
  Max grid size in x:2147483647, y:65535, z:65535
  Total Global Memory: 16 GB
  Peak Memory Bandwidth:  448 GB/s
```

```
int numOfDevices;

cudaGetDeviceCount(&numOfDevices);

printf("\n\n * You have %d CUDA-enabled devices *\n\n",numOfDevices);

//enumerate though devices
for (int i = 0; i < numOfDevices; i++) {

cudaDeviceProp properties;
cudaGetDeviceProperties(&properties, i); //puts all properties in the properties variable
printf(" === Information for device number %d ===\n", i);

//Properties can be accessed by their name

printf(" Device name: %s\n", properties.name);
printf(" Compute Capability: %d.%d\n", properties.major, properties.minor);
printf(" Maximum threads per SM: %d\n", properties.maxThreadsPerMultiProcessor);

}
```

Using nvidia-smi and property query

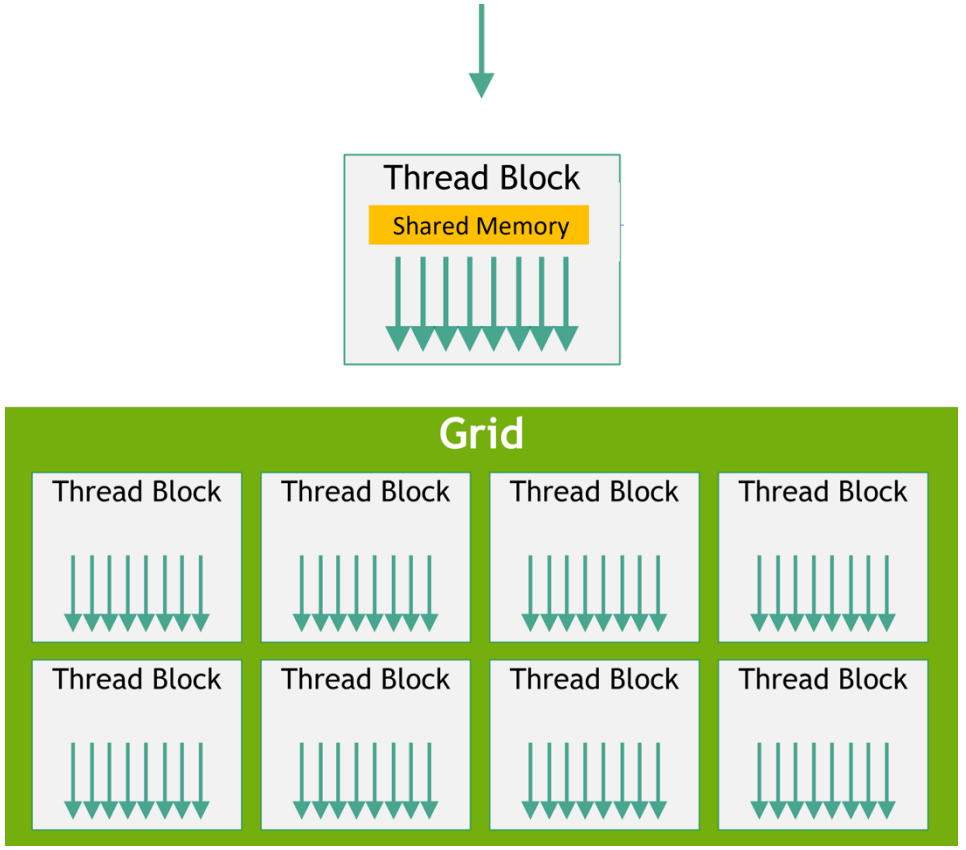
```
c196-082[rtx](350)$ nvidia-smi
Wed Apr  1 16:49:24 2026
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.113.01             Driver Version: 535.113.01   CUDA Version: 12.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Quadro RTX 5000                On  | 00000000:02:00.0 Off |                  Off |
|  0%   31C    P8              11W / 230W |      0MiB / 16384MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   1  Quadro RTX 5000                On  | 00000000:03:00.0 Off |                  Off |
|  0%   31C    P8               2W / 230W |      0MiB / 16384MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   2  Quadro RTX 5000                On  | 00000000:82:00.0 Off |                  Off |
|  0%   31C    P8               5W / 230W |      0MiB / 16384MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   3  Quadro RTX 5000                On  | 00000000:83:00.0 Off |                  Off |
|  0%   31C    P8               6W / 230W |      0MiB / 16384MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

Execution Model & Hardware Mapping

Threads in a warp execute in-step

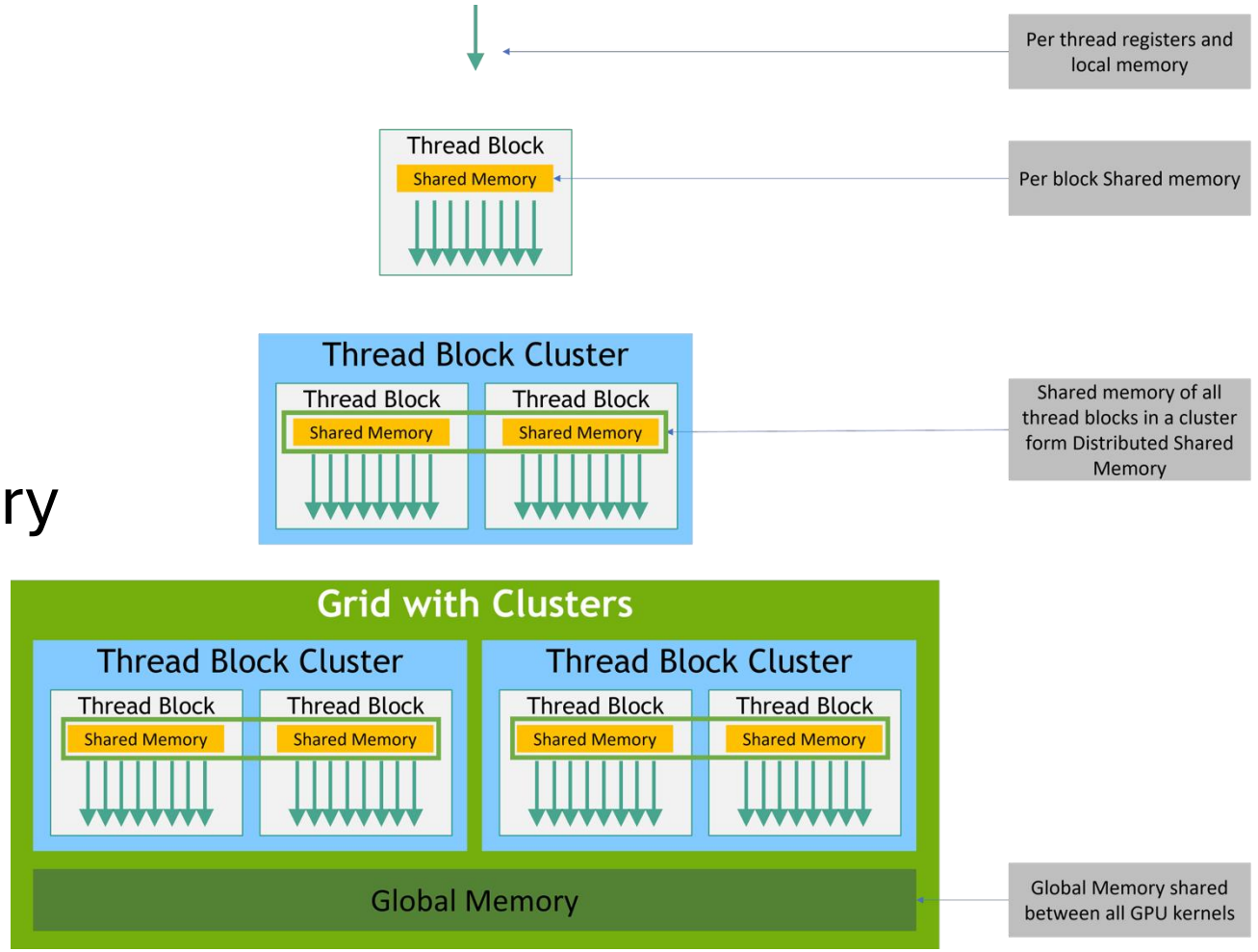
Only guarantee of warp concurrency

- Threads are mapped to CUDA cores
- Blocks mapped to SMs
- Grid mapped to the GPU



Memory Hierarchy

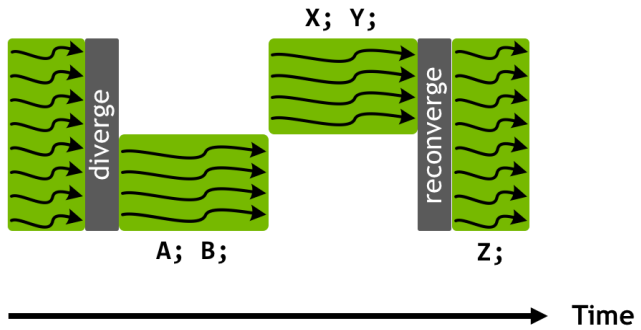
- High latency in data transfer hidden with multiple levels of memory
- Registers
- L1 local cache / Shared Memory
- L2 shared by all SMs – intermediate in Host communication
- Global Memory (DRAM)



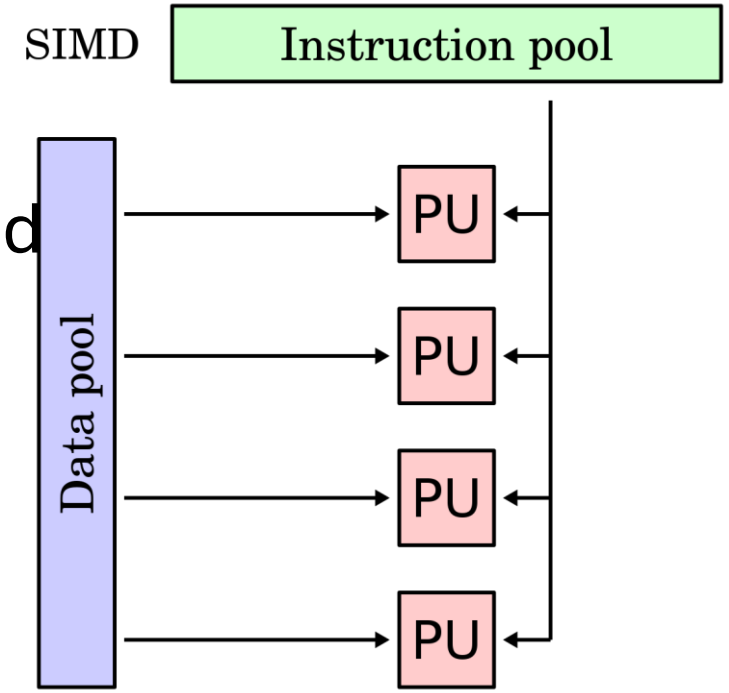
Warps and SIMT Execution Pipeline

- Threads in a warp execute in-step
- Only guarantee of warp concurrency
- Divergent branches (if; else) should be avoided

```
if (groupThreadID.x < 4){
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



From Nvidia Technical Blog



PU – Processing Unit, here a GPU thread

https://en.algorithmica.org/hpc/simd/

Migrating the data

- Data is initialized on the CPU, then copied to the GPU
- CPU ←→ GPU transfers are slow, keep them to the minimum

- Ways to transfer the data:
  - Allocate space on the GPU, copy from CPU to GPU (and back)
  - Use Unified (Managed) Memory

Data allocation and movement

allocate memory on the device

```
cudaMalloc(device_pointer,  size)
```

copy data between host and device

```
cudaMemcpy(dest_ptr,  src_ptr, size, direction)
```

Data copy directions

0: cudaMemcpyHostToHost
1: cudaMemcpyHostToDevice
2: cudaMemcpyDeviceToHost
3: cudaMemcpyDeviceToDevice

```
int *array, *d_array;

cudaMalloc(&d_array, N * sizeof(int));

cudaMemcpy(d_array, array, N*sizeof(int), cudaMemcpyHostToDevice);
```

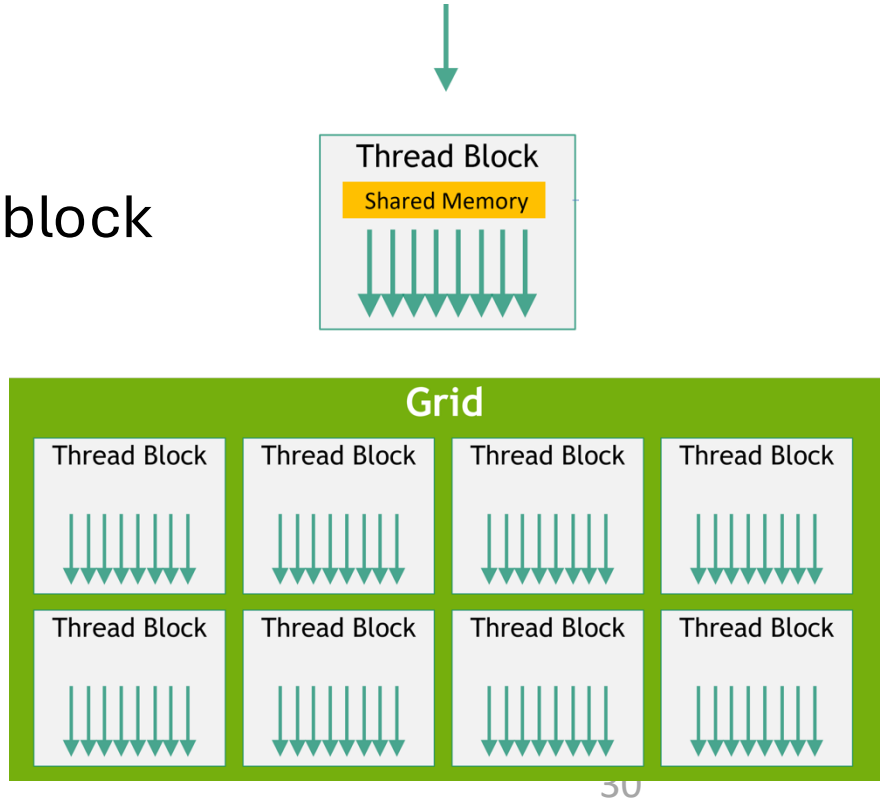
Kernel – a GPU function

- Kernels declared as normal functions with ***__global__*** attribute
- ***__global__*** *myKernel*(kernel parameters){ …kernel body…};
- ***__global__*** makes the function callable from host and device
- Called with information about # of blocks and threads/block
- *myKernel*<<< ***# Blocks*** , ***Threads per block*** >>>(parameters);

Kernel configuration

- Chevron syntax <<< >>> specifies the resources provided to the kernel
- Two required positional parameters <<<**GridSize**, **BlockSize** >>>

- **GridSize** = Number of thread blocks
- **BlockSize** = Number of threads in a block



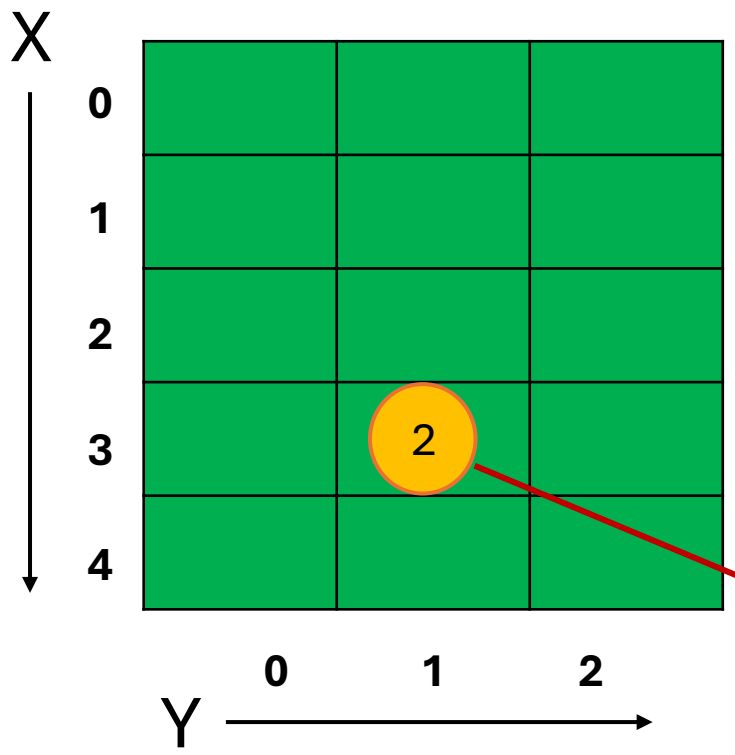
Kernel configuration

- Two-tier hierarchy:
  - **Grid** of blocks
  - **Block** of threads
- Why the division?
  - Entire block assigned to one SM – shared memory, synchronization
  - Grid specifies how many blocks we want to launch
  - Since block is mapped to SM, it will only start when resources are available

Gridsize and blocksize

- Grid size and block size are dim3 (x, y, z): can be 1D, 2D or 3D
- Block size limited by max number of threads 1024. Block size must be the multiple of 32 (warp size).
- Grid size imposes limit in each direction, no limit for total grid size (x: 2^{31}-1, y: 65535, z: 65535)
- Multidimensional grid helps to map to multidimensional data

Indexing the threads




```
dim3 GridSize(5,3) dim3 BlockSize(512)
someKernel<<<GridSize,BlockSize>>>();
```

Position on the grid = GridSize.x *GridIdx.y + BlockId.x → 5 x 1 + 3 = 8

Position in the block = ThreadIdx.x

Global Id = (GridSize.x *GridIdx.y + BlockId.x) *BlockSize.x + ThreadIdx.x

Converting CPU code to GPU code

A simple serial CPU function to square an array
Loop over each elements, each iteration squares the value at array[i] and writes it back to a[i]

```
void squareArray(int* array, int N){

for (int i=0; i < N; i++){
        array[i] = array[i] * array[i];
    }
}
```

Mapping threads to loop iterations

```
void squareArray(int* array, int N){

for (int i=0; i < N; i++){
        array[i] = array[i] * array[i];
    }
}
```

```
__global__ d_squareArray(int* d_array, int N){

    int tid = BloxkSize.x * BlockIdx.x + ThreadIdx.x;

    // don't go out of bounds
    if (tid < N){
    // each thread maps to a loop iteration
    d_array[i] = d_array[i] * d_array[i];
    }
}
```

Data allocation and movement

```
//host allocation and initialization

const int N = 1024;

int *array = (int *)malloc(sizeof(int) * N);

for (int i = 0; i < N; i++) {array[i] = i;}

//device allocation and copy

int *d_array;

cudaMalloc(&d_array, N * sizeof(float));

cudaMemcpy(d_array, array, N*sizeof(int), cudaMemcpyHostToDevice);
```

Running the function / kernel

```
// CPU
squareArray(array, N);

int blockSize = 512;

int gridSize = N/blockSize;

//GPU
d_squareArray <<<gridSize, blockSize>>>(d_array, N);

cudaMemcpy(array, d_array, N*sizeof(int), cudaMemcpyDeviceToHost);

//deallocate the memory
free(array);

cudaFree(d_array);
```

Kernel structure

```
__global__ void arrayAdd(float *d_a, float *d_b, float *d_out, int size)
{
        // Calculate the global thread ID

        int tid = blockIdx.x * blockDim.x + threadIdx.x;

        // Make sure we don't go out of bounds

        if (tid < size) {
           d_out[tid] = d_a[tid] + d_b[tid];
           }
}
```

Hands on exercise no. 1

Source code in 1_FirstKernel/arrayAdd.cu

Simple program which:

- Initializes data on the CPU
- Copies it to GPU
- Adds arrays together
- Measures execution time of the kernel
- Copies the result back to CPU
- Measures bandwidth of the data transfers
- Shows how to perform CUDA error checking

Check for errors

```
cudaError_t err;

err = cudaGetLastError();
if (err != cudaSuccess)
{
   printf("CUDA Error:%s\n",cudaGetErrorString(err));
   return -1;
}
```

CUDA events

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);

... GPU code to time ...

cudaEventRecord(stop);
cudaEventSynchronize(stop);
cudaEventElapsedTime(&time, start, stop);
```

- More precise than CPU timers for measuring GPU execution time
- cudaEventRecord puts a time stamp into a given event
- cudaEventElapsedTime measures time between the events in ms.

Sample output

- Effective bandwidth: (total bytes read + written)/execution time
- Notice the warm-up run before the actual timing

```
./arrayAdd
Copy H->D took 0.144992 ms
CUDA kernel launch with 40 blocks of 256 threads
Kernel warm up time: 36.640991 ms
Matrix addition took 0.004992 ms
Effective Bandwidth (GB/s): 24.038462
Copy D->H took 0.015424 ms
```

Exercise #1: Using grid-stride loop

- The size of the input array can be larger than the number of available threads
- Threads can be reused to process multiple elements
- Grid-stride loop ensures coalesced memory

```
for (int i = blockIdx.x * blockDim.x + threadIdx.x;
          i < size;
          i += blockDim.x * gridDim.x)
```

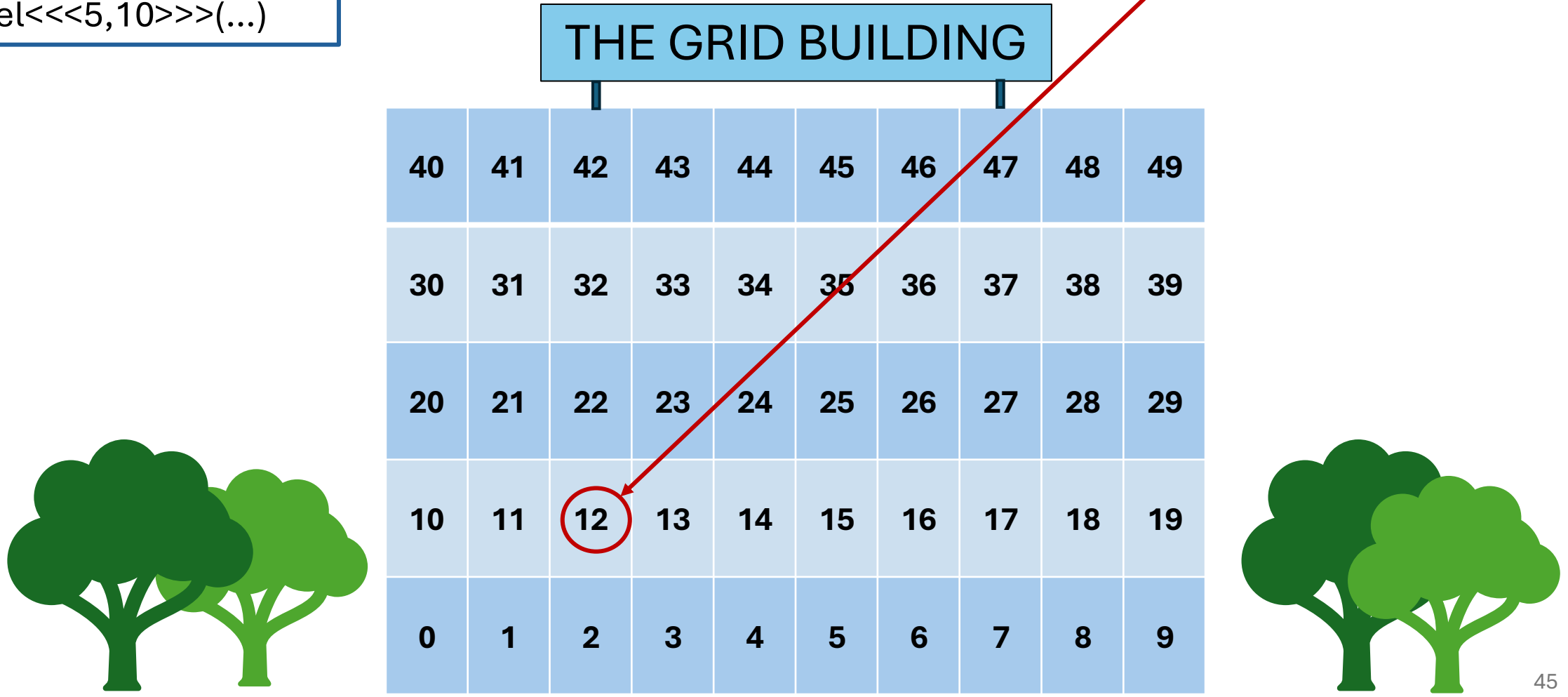
Quick review

- **Kernel** = GPU function
- **Host** = CPU, **device** = GPU
- Variables **blockDim.x**, **blockIdx.x** and **threadIdx.x** are known at runtime and characterize each thread
- We can have **blockDim.x/y/z** (same for blockIdx and threadIdx) since these variables can be 1D,2D or 3D. Memory is always linear and 3D coordinates can always be converted to 1D. We will only work in 1D and only care about ”.x”.
- **blockDim.x** – (x) dimension of the thread block
- **blockIdx.x** – id of the block the specific thread is in
- **threadIdx.x** – id (position) of the given thread within its block
- Kernel is given its resource (no. of blocks and threads) using the <<<…>>> syntax.

Hypothetical scenario of 5 blocks and 10 threads per block (not correct numbers for CUDA)

Building = Grid
Floor = blockIdx.x
Apartment = threadIdx.x
kernel<<<5,10>>>(...)

Example (indexing from 0) 3rd apartment on floor 1:
blockIdx.x = 1
threadIdx.x = 2 (because 0,1,2)
Global_id = full floors below + my number on my floor
Global_id = 10 * 1 + 2



Using nvidia-smi

```
nvidia-smi #general information

nvidia-smi –q #detailed GPU query

nvidia-smi -q -d POWER -i 0 #detailed power query on GPU no. 0

nvidia-smi -q -d TEMP -i 2 #detailed temperature query on GPU no. 2

nvidia-smi –h #display help

### Admin tools ###

sudo nvidia-smi --lock-gpu-clocks=1980 # set fixed GPU clock

sudo nvidia-smi –power-limit=900 # set power limit
```

https://nvidia.custhelp.com/app/answers/detail/a_id/3751/~/useful-nvidia-smi-queries
https://docs.nvidia.com/deploy/nvidia-smi/

Profiling tools

- For more insight we can use profiling:

- **Nsight Systems** – for profiling entire code
- **Nsight Compute** – profiling specific kernel, detailed report

- Repots can be viewed in GUI on any system

- https://developer.nvidia.com/nsight-systems/get-started
- https://developer.nvidia.com/nsight-compute

Using Nsight Systems

```
nsys profile -o arrayAddprof --stats true ./arrayAdd
```

- The command will profile the code, create arrayAddprof.nsys-rep report file and print statistics to the terminal.
- To display them again from the report, use:

```
nsys stats arrayAddprof.sqlite
```

Example output

```
[6/8] Executing 'cuda_gpu_kern_sum' stats report

 Time (%)  Total Time (ns)  Instances  Avg (ns)  Med (ns)  Min (ns)  Max (ns)  StdDev (ns)              Name
 --------  ---------------  ---------  --------  --------  --------  --------  -----------  ----------------------------------
    100.0            3,360          2   1,680.0   1,680.0     1,664     1,696         22.6  arrayAdd(int *, int *, int *, int)

[7/8] Executing 'cuda_gpu_mem_time_sum' stats report

 Time (%)  Total Time (ns)  Count  Avg (ns)  Med (ns)  Min (ns)  Max (ns)  StdDev (ns)          Operation
 --------  ---------------  -----  --------  --------  --------  --------  -----------  ----------------------------
     70.1            6,080      2   3,040.0   3,040.0     2,912     3,168        181.0  [CUDA memcpy Host-to-Device]
     29.9            2,592      1   2,592.0   2,592.0     2,592     2,592          0.0  [CUDA memcpy Device-to-Host]

[8/8] Executing 'cuda_gpu_mem_size_sum' stats report

 Total (MB)  Count  Avg (MB)  Med (MB)  Min (MB)  Max (MB)  StdDev (MB)          Operation
 ----------  -----  --------  --------  --------  --------  -----------  ----------------------------
      0.080      2     0.040     0.040     0.040     0.040        0.000  [CUDA memcpy Host-to-Device]
      0.040      1     0.040     0.040     0.040     0.040        0.000  [CUDA memcpy Device-to-Host]
```

Lecture Outline

- Using Nsight Compute
  - What is a roofline plot
- Memory access patterns (strided, random) and their performance
- CUDA Streams – making sure we utilize entire GPU
  - Concurrent kernels
  - Asynchronous Memory Transfers
  - Visualization in Nsight Systems
- Shared Memory – optimizing data reuse and memory access
- Warp-level functions – using thread registers for better performance

Using Nsigh Compute

Command below tells Nsight Compute to profile arrayAdd kernel in the executable arrayAdd. Report will be saved in **arrayAddprof.ncu-rep.** Report name is specified with –*o* option and the .ncu-rep extension added automatically.

--set option specifies which metrics to collect.

```
ncu -o arrayAddprof --set full --kernel-name=arrayAdd ./arrayAdd
```

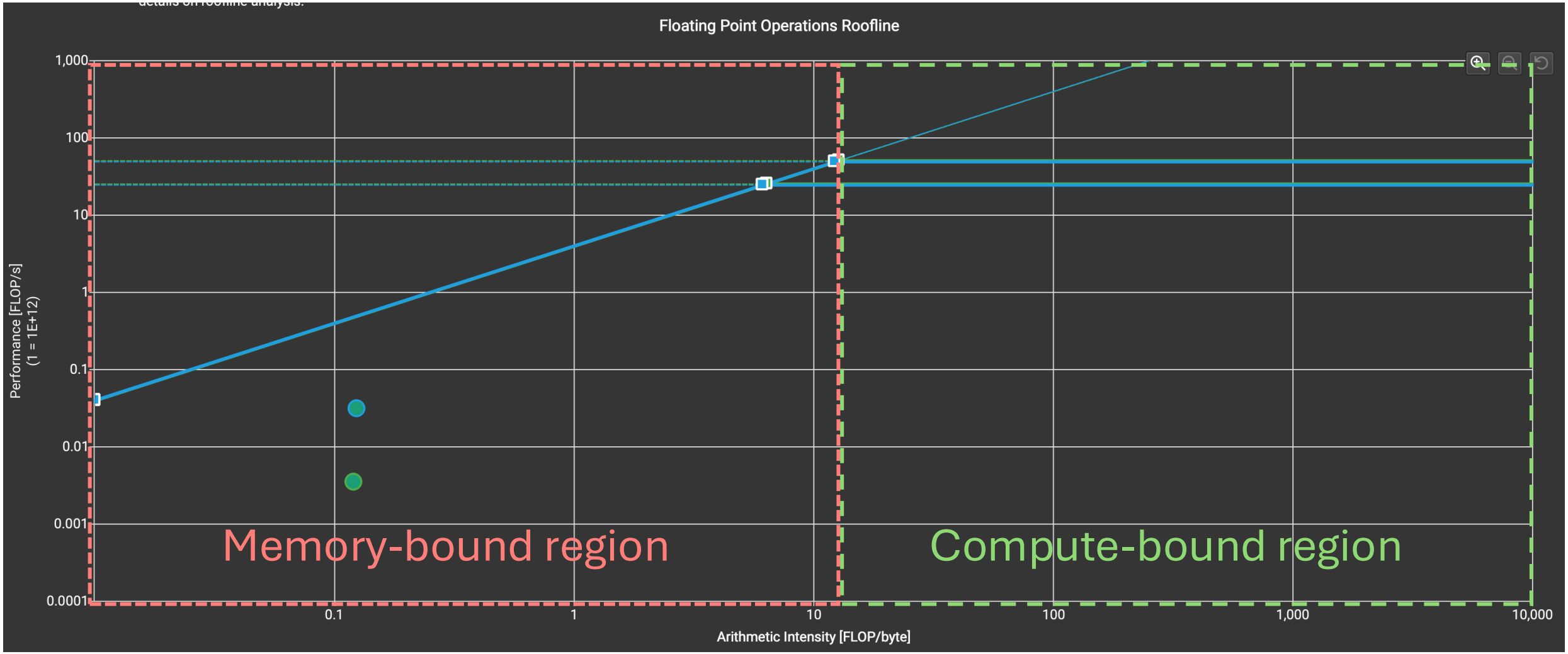
- Use --import mode to display report details in the terminal

```
ncu --import arrayAddprof.ncu-rep
```

```
Section: Memory Workload Analysis
------------------------- ----------- ------------
Metric Name               Metric Unit Metric Value
------------------------- ----------- ------------
Memory Throughput             Gbyte/s       25.17
Mem Busy                            %        0.66
Max Bandwidth                       %        0.63
L1/TEX Hit Rate                     %           0
L2 Compression Success Rate         %           0
L2 Compression Ratio                            0
L2 Hit Rate                         %       36.81
Mem Pipes Busy                      %        1.06
------------------------- ----------- ------------
```

```
Section: GPU Speed Of Light Throughput
---------------------- ----------- ------------
Metric Name            Metric Unit Metric Value
---------------------- ----------- ------------
DRAM Frequency                 Ghz        2.58
SM Frequency                   Ghz        1.51
Elapsed Cycles               cycle       4,298
Memory Throughput                %        0.77
DRAM Throughput                  %        0.73
Duration                        us        2.85
L1/TEX Cache Throughput          %        2.14
L2 Cache Throughput              %        1.50
SM Active Cycles             cycle      662.30
Compute (SM) Throughput          %        1.09
---------------------- ----------- ------------
```

Google time!
Keywords: roofline plot, memory-bound, compute-bound



Memory or compute bound?

$$Arithmetic\ Intensity\ (AI) = \frac{Total\ floating-point\ ops}{Total\ bytes\ moved} = \frac{FLOPs}{byte}$$

$$Ridge\ Point\ (RP) = \frac{Peak\ compute\ BW}{Peak\ memory\ BW} = \frac{FLOPs}{sec} * \frac{sec}{byte}$$

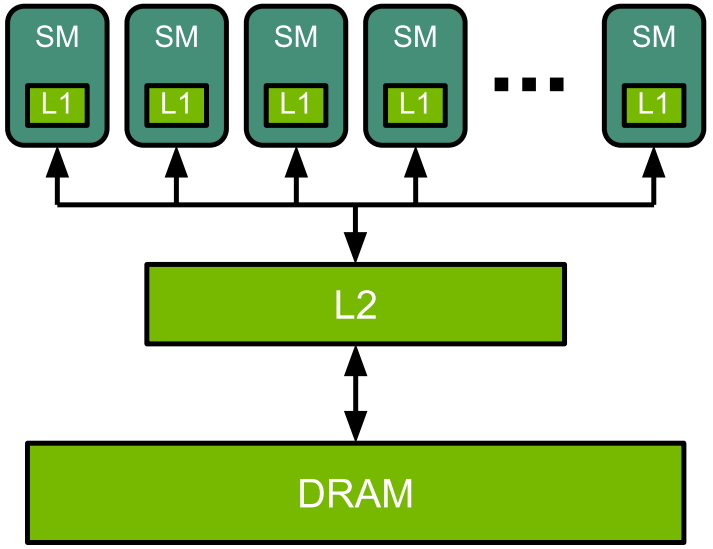
$$Memory\ bound\colon\ AI < RP$$

$$Compute\ Bound\colon\ AI > RP$$

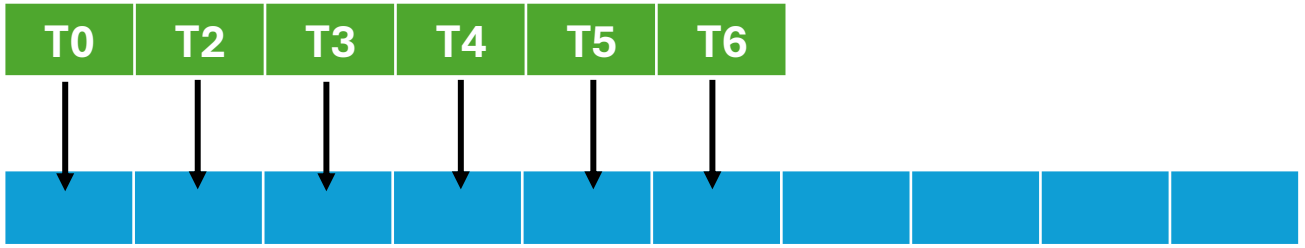
Review on memory types

Location	Capacity	Bandwidth (GB/s)	Latency (ns)
L1 cache/shared memory	192 KB/SM	~19,000	~20
L2 cache	40 MB	~4,000	~150
HBM (global)	80 GB	~1,500	~400

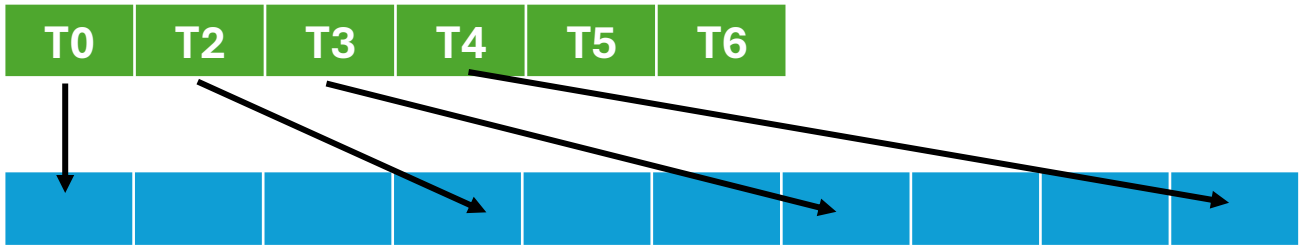
Approximate data for A100 from NVIDIA GTC “How GPU Computing Works”



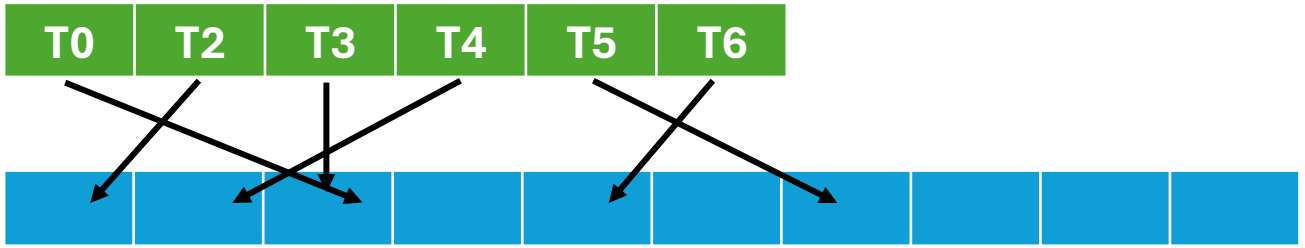
Global memory access patterns



Coalesced memory access pattern.
Consecutive threads load consecutive memory locations. Loads can be done in one transaction – most efficient.



Strided memory access pattern.
Constant offset, here stride size is 2.
Performance decreases with stride size.



Random memory access pattern.
Worst for performance.

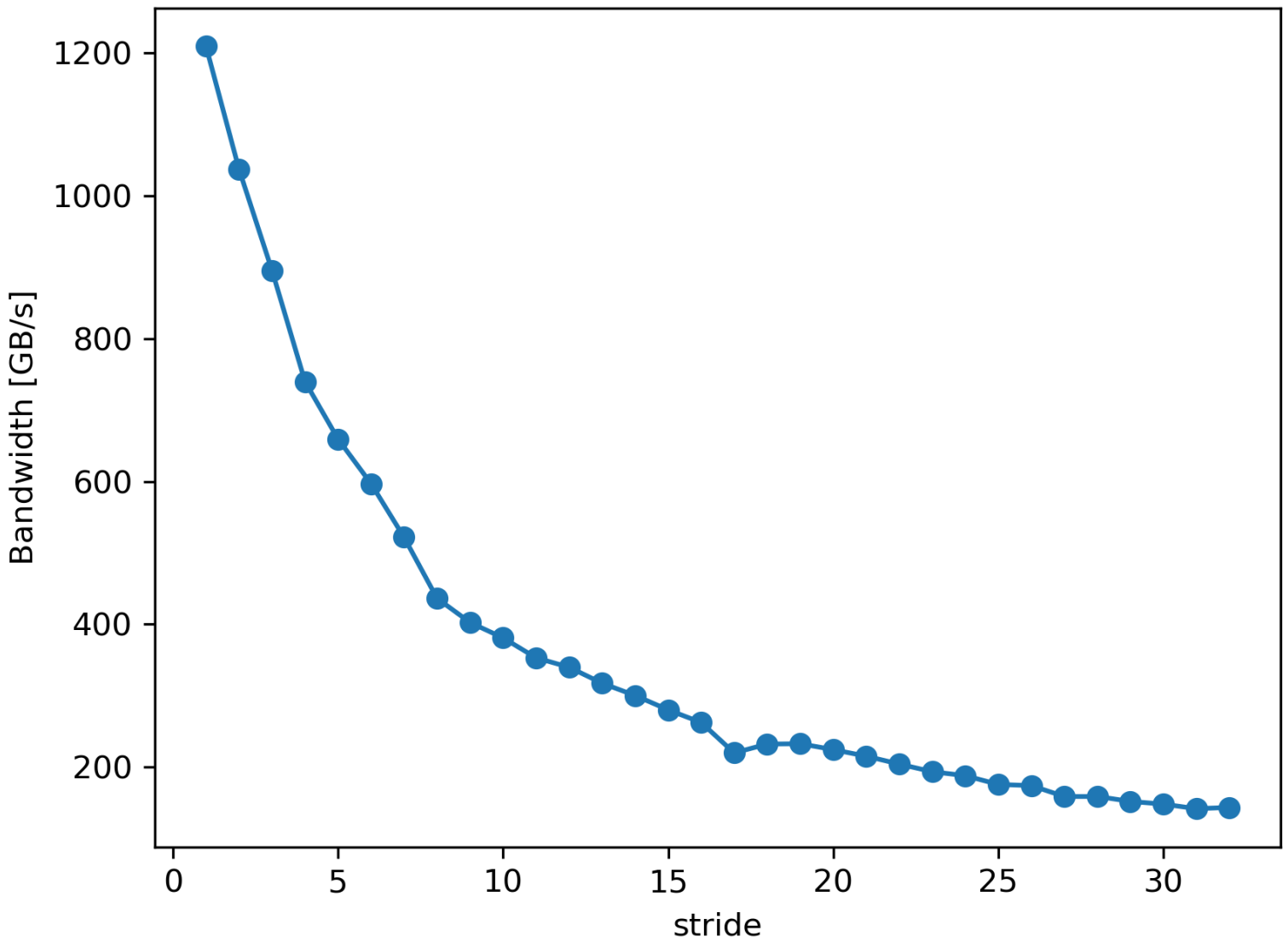
Hands-on exercise

```
idev –p rtx
module load cuda
cd 2_MemoryAccess
nvcc –o stride stride.cu
./stride
```

Strided memory access kernel

```
__global__ void stridedMemAccess(float *d_in, int stride){

        int tid = (blockDim.x * blockIdx.x + threadIdx.x) * stride;

        d_in[tid] = d_in[tid] + stride;
}

for (int stride = 1; stride <= 32; stride++) {
        stridedMemAccess<<<gridSize, blockSize>>>(d_in,stride);
}
```

	tid = 0	tid = 1	tid = 2	tid = 3	tid = 4	tid = 5
stride=1	0	1	2	3	4	5
stride=2	0	2	4	6	8	10
stride=5	0	5	10	15	20	25



Processing 1310720 fp32 elements
Stride: 1, Bandwidth (GB/s) 1020.809984
Stride: 2, Bandwidth (GB/s) 1067.361536
Stride: 3, Bandwidth (GB/s) 880.860224
Stride: 4, Bandwidth (GB/s) 753.287360
Stride: 5, Bandwidth (GB/s) 672.854208
Stride: 6, Bandwidth (GB/s) 580.992896
Stride: 7, Bandwidth (GB/s) 513.604992
Stride: 8, Bandwidth (GB/s) 441.617248
Stride: 9, Bandwidth (GB/s) 400.097664
Stride: 10, Bandwidth (GB/s) 377.946944
Stride: 11, Bandwidth (GB/s) 351.588000
Stride: 12, Bandwidth (GB/s) 334.026496
Stride: 13, Bandwidth (GB/s) 314.774240
Stride: 14, Bandwidth (GB/s) 289.469952
Stride: 15, Bandwidth (GB/s) 275.824928
Stride: 16, Bandwidth (GB/s) 260.270048
Stride: 17, Bandwidth (GB/s) 251.867792
Stride: 18, Bandwidth (GB/s) 242.366848
Stride: 19, Bandwidth (GB/s) 231.739744
Stride: 20, Bandwidth (GB/s) 226.768176
Stride: 21, Bandwidth (GB/s) 215.154304
Stride: 22, Bandwidth (GB/s) 206.088048
Stride: 23, Bandwidth (GB/s) 192.526432
Stride: 24, Bandwidth (GB/s) 183.368768
Stride: 25, Bandwidth (GB/s) 174.855920
Stride: 26, Bandwidth (GB/s) 169.343680
Stride: 27, Bandwidth (GB/s) 161.259840
Stride: 28, Bandwidth (GB/s) 155.372208
Stride: 29, Bandwidth (GB/s) 150.796128
Stride: 30, Bandwidth (GB/s) 148.810176
Stride: 31, Bandwidth (GB/s) 144.416032
Stride: 32, Bandwidth (GB/s) 143.467600

Hands-on exercise

Complete the kernel demonstrating random memory access

2_MemoryAccess

open random.cu

Solution in solution_random.cu

```
int N = gridSize * blockSize; // total number of threads = N

int *h_rng = (int *)malloc(N * sizeof(int)); // array of random numbers

cudaMalloc(&d_rng, N * sizeof(int)); // array of random numbers

for (int i = 0; i < N; i++){h_rng[i] = i;}

unsigned seed = … // time-based seed

shuffle(h_rng, h_rng + N, default_random_engine(seed));

cudaMemcpy(d_rng, h_rng, N*sizeof(int), cudaMemcpyHostToDevice);
```

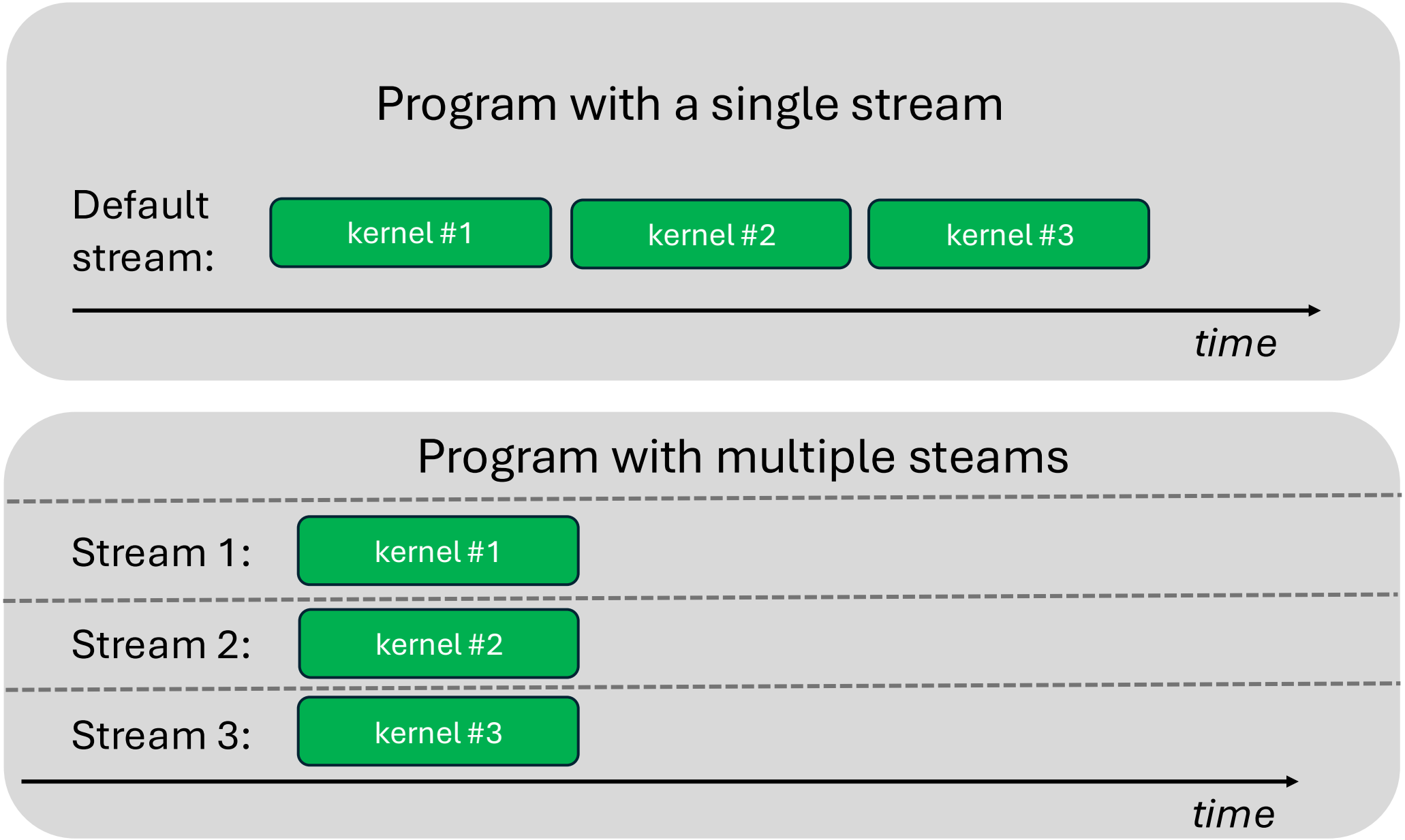
```
__global__ void randomMemAccess(float *d_in, int *d_rng){


// assign one random number from d_rng to each thread
// each thread operates on one element of d_in[n], index "n" determined by its assigned random number
        d_in[n] = d_in[n] + 1.0;
}
```

CUDA streams – even more parallelism

- Kernel launches are **asynchronous** with respect to the host (CPU). CPU issues the instruction to start the kernel and immediately moves on the the next instruction.
- Data transfer instructions (*cudaMemcpy*) are **synchronous** or blocking with respect to the host. After issuing a *cudaMemcpy* call, the CPU waits for the data transfer to complete before moving to the next instruction.
- Data transfer can be made asynchronous *with* *cudaMemcpyAsync* (requires pinned memory).

CUDA Streams

- CUDA streams are sequences of instructions issues by the host (CPU) to be executed on the device (GPU).
- Operations within a single stream will execute in sequentially in order the were issued to the device.
- A single program can have multiple streams.
- Operations in one stream can run concurrently with operations from a different stream (if sufficient resources are available)

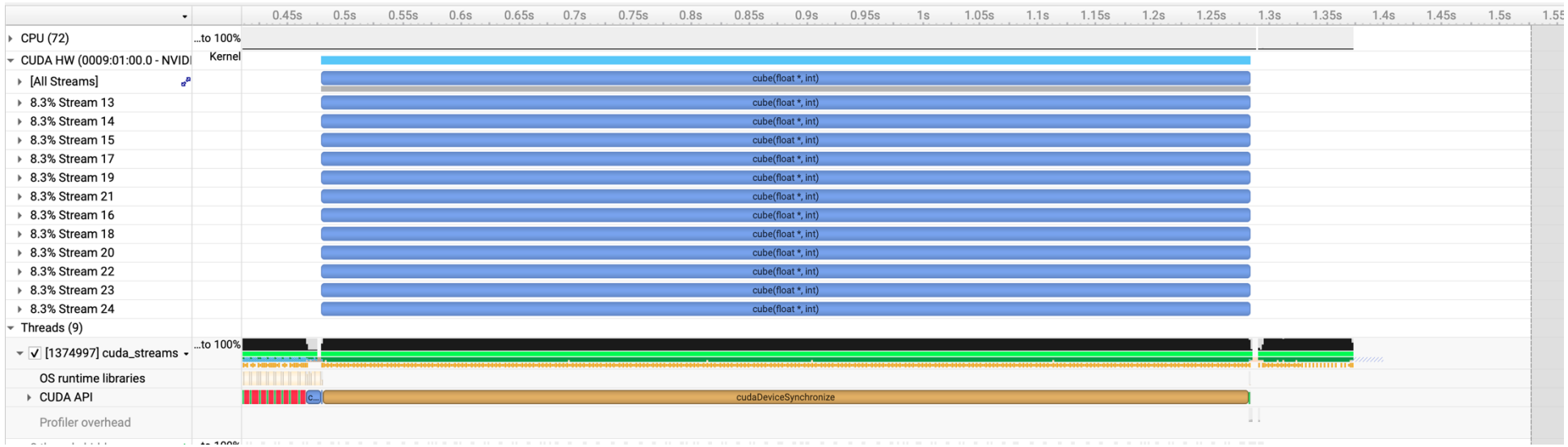

Program with a single stream
Default stream:
kernel #1
kernel #2
kernel #3
time
Program with multiple steams
Stream 1:
kernel #1
Stream 2:
kernel #2
Stream 3:
kernel #3
time

Using streams

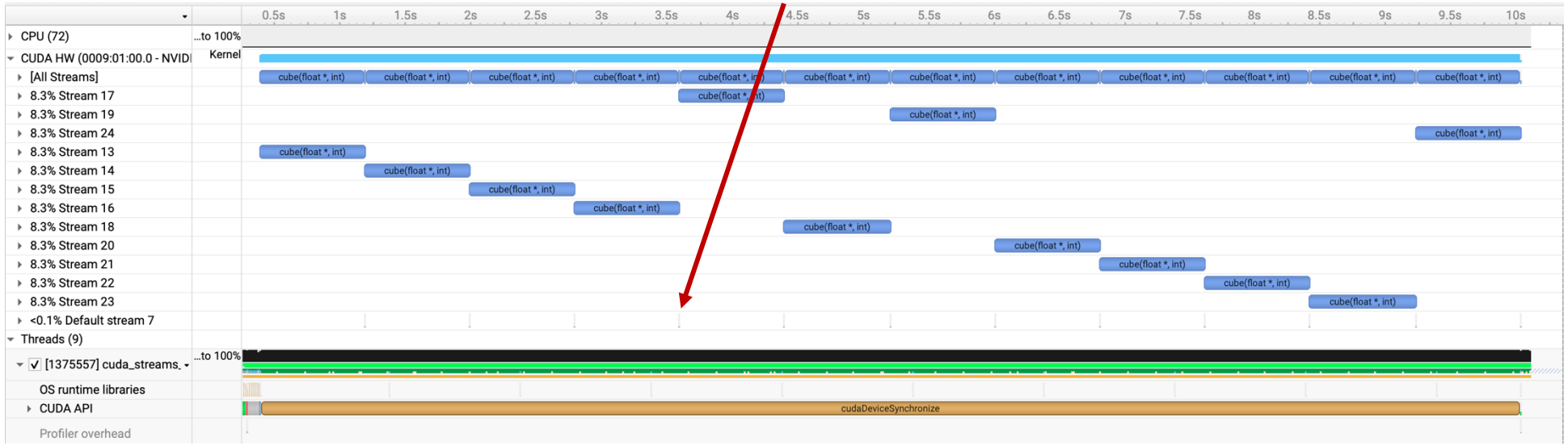
- So far, we have been using the **default stream** (used when no stream is specified).
- No operations in any stream can begin until all previously issued calls in the default stream have finished.
- Operations in the default stream will not begin until all operations in all other streams have finished. This can cause serialization between different streams.

```
cudaStream_t stream1;
cudaStreamCreate(&stream1);

kernel<<<gridSize,blockSize,0,stream1>>>(…);

kernel<<<gridSize,blockSize>>>(…); #default

cudaStreamSynchronize(&stream1);
cudaStreamDestroy(&stream1);
```

```
idev –p rtx
module load cuda
cd 3_CUDAStreams
nvcc –o cuda_streams cuda_streams.cu
nsys profile –o cuda_streams_prof ./cuda_streams
```



Default stream

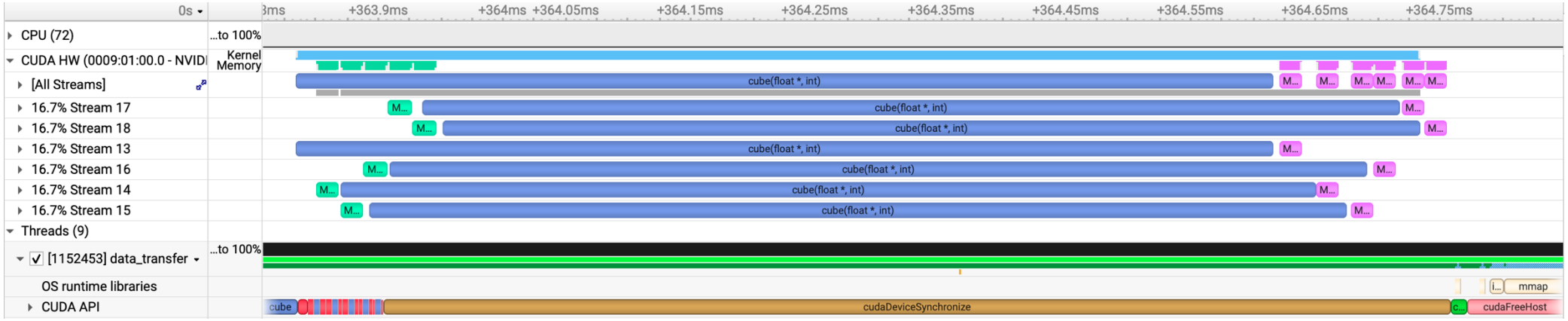


Asynchronous data copies

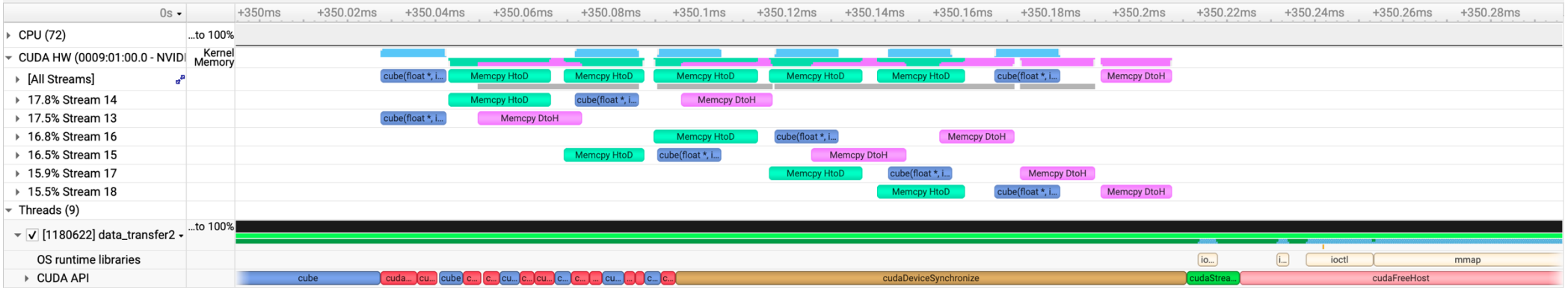
- Similarly to overlapping kernels from different streams, we can overlap data copies with kernel execution.
- To achieve that we use *cudaMemcpyAsync()* call.
- Requirements for concurrent transfer:
- Kernel call and the memory copy call need to be in different streams
- Host memory needs to be pinned (page-locked)

```
3_CUDAStreams
nvcc –o async_copy async_copy.cu
nsys profile –o async_copy_prof ./async_copy
```

Kernel configuration <<<4, 128>>>



Kernel configuration<<<128, 256 >>>

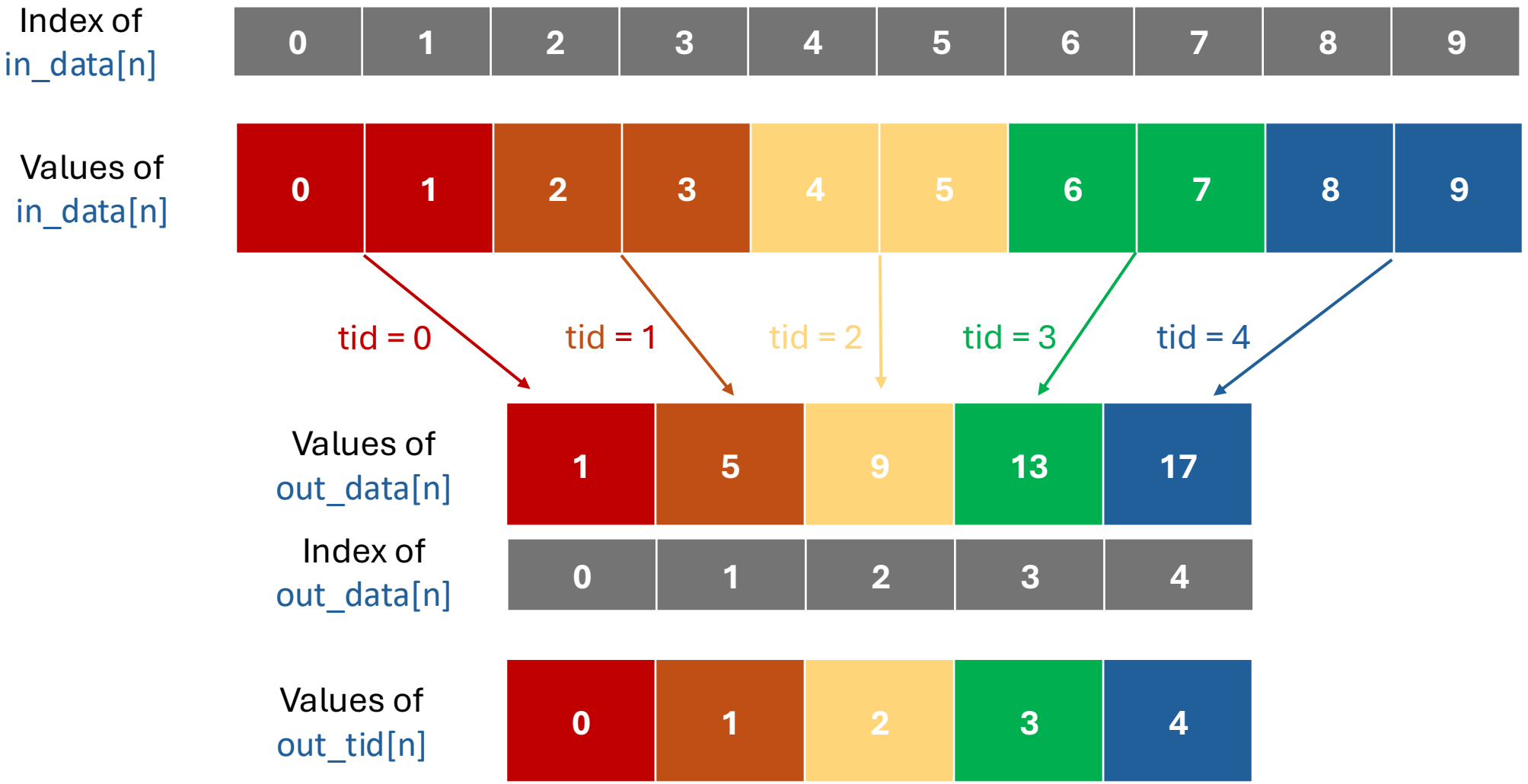


Homework #1

- You are given input array of size 2*N and two output arrays of size N.
- Input: *in_data*; output *out_data*, *out_tid*
- Each thread adds together two consecutive elements of *in_data* and writes the result to *out_data*. Elements added by threads do not overlap (if one thread added element 0 and 1, next thread will add elements 2 and 3). Threads are assigned to elements sequentially, starting with the thread with the lowest global ID.
- If the size of *in_data* is larger than total number of threads, use grid-size stride and start the sequence again from tid 0.
- Use the kernel configuration and N provided in cudaHW1.cu
- Homework: finish the kernel

Example with kernel configuration <<<1,32>>> and N = 96
tid = 0 : out_data[0] = in_data[0] + in_data[1]; out_tid[0] = 0
tid = 1 : out_data[1] = in_data[2] + in_data[3]; out_tid[1] = 1
tid = 2 : out_data[2] = in_data[4] + in_data[5]; out_tid[2] = 2
.
.
.
tid = 31 : out_data[31] = in_data[62] + in_data[63]; out_tid[31] = 31
tid = 0 : out_data[32] = in_data[64] + in_data[65]; out_tid[32] = 0
tid = 1 : out_data[33] = in_data[66] + in_data[67]; out_tid[33] = 1

Output values shoul be out_data[i] == 4 * i + 1


Index of
in_data[n]
0
1
2
3
4
5
6
7
8
9
Values of
in_data[n]
0
1
2
3
4
5
6
7
8
9
tid = 0
tid = 1
tid = 2
tid = 3
tid = 4
Values of
out_data[n]
1
5
9
13
17
Index of
out_data[n]
0
1
2
3
4
Values of
out_tid[n]
0
1
2
3
4

TAP – TACC Analysis Portal

https://tap.tacc.utexas.edu

Submit New Job

System: Frontera

Application: DCV remote desktop

Project: ASC25001

Queue: rtx

Nodes: 1 Tasks: 16

Options

Job Name: test

Time Limit: H:M:S (default 2:0:0)

Reservation: coe-rtx-apr14

VNC Desktop Resolution: WIDTHxHEIGHT

Submit Utilities

System Status

System	Status	Utilization	Job Count
Frontera Details	Open	94%	Running: 338 Waiting: 98
Lonestar6 Details	Open	81%	Running: 287 Waiting: 592
Stampede3 Details	Open	95%	Running: 529 Waiting: 286
Vista Details	Open	100%	Running: 160 Waiting: 248

Past Jobs

test tap vista res	04/10/2026	Details	Resubmit
Test TAP	04/10/2026	Details	Resubmit
test tap on vista	04/10/2026	Details	Resubmit
test tap on vista	04/10/2026	Details	Resubmit
test tap on vista	04/10/2026	Details	Resubmit

Waiting for resources

TAP Job Status

Job: test
Status: PENDING
Refresh: in 55 seconds
Message:

```
TAP: Your script has been submitted to Frontera but it is not yet running.

`sinfo -p rtx` output is:
   PARTITION          AVAIL NODES(A/I)
   rtx                up    23/49
```

Check Status | End Job | Show Output | Back to Jobs

Node is ready

TAP Job Status

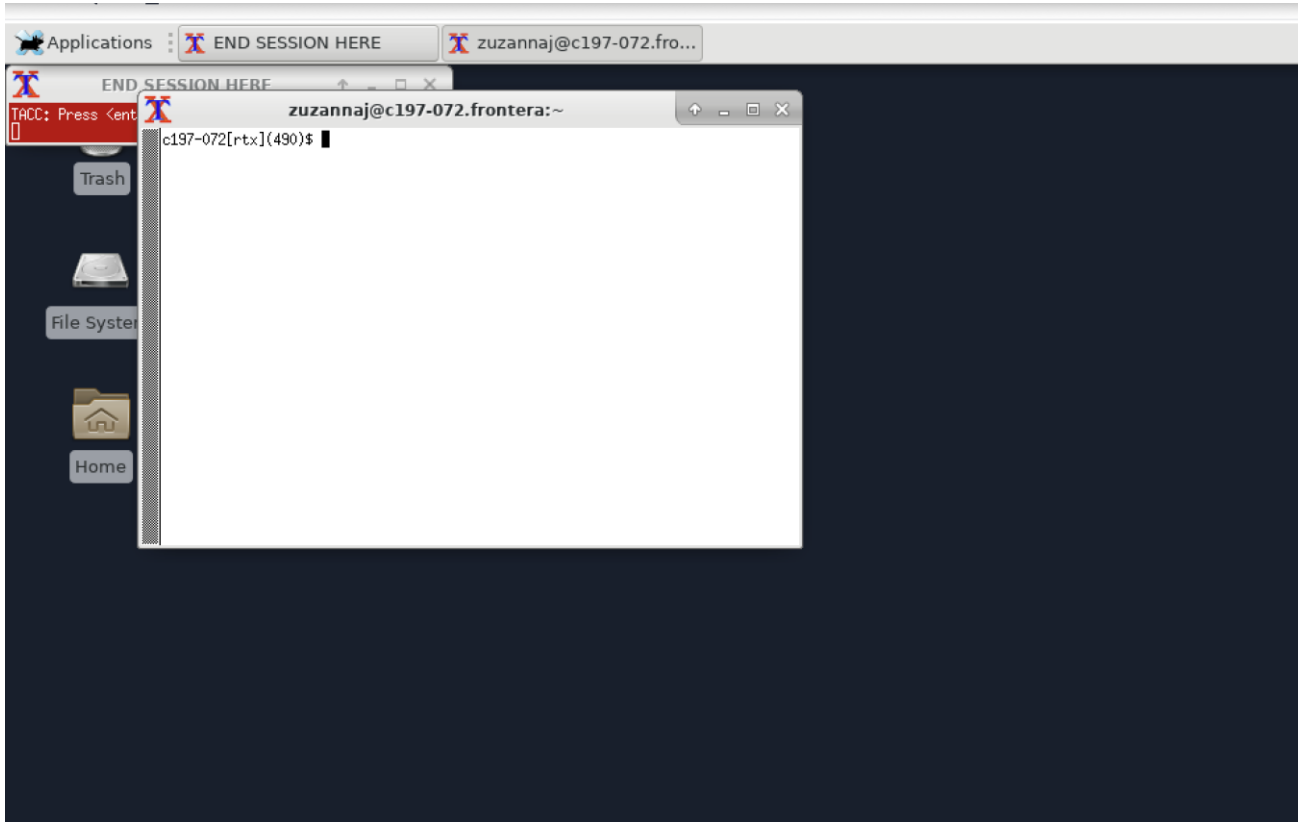
Job: test
Status: RUNNING
Start: 04/14/2026 11:37
End: 04/14/2026 13:37
Refresh: in 868 seconds
Message:

```
TAP: Your session is running at https://frontera.tacc.utexas.edu:60233
```

Connect | End Job | Show Output | Back to Jobs

Click here to connect


Sign in with your credentials
Username
Password


Applications
END SESSION HERE
zuzannaj@c197-072.fro...
END SESSION HERE
TACC: Press <ent
zuzannaj@c197-072.frontera:~
c197-072[rtx](490)$
Trash
File Syste
Home

Shared memory

- On-chip memory, x100 faster than global memory
- Manually managed part of L1 cache
- Visible to all threads within the same block
- When to use shared memory:
  - Communication within the block
  - Combining multiple writes to global memory into a single transaction
  - Data re-use within the block
- Can cause race condition, need for explicit synchronization barrier within the block with *__syncthreads()*

Using shared memory

- **Static** shared memory - known at compile time – declared within the kernel

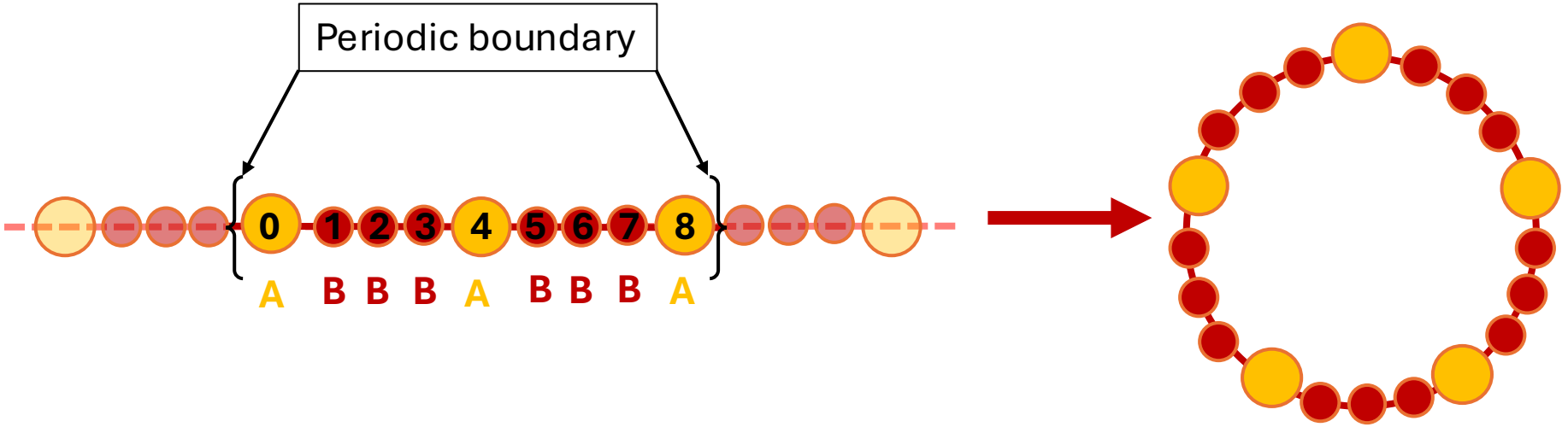
```
__global__ static_shered_mem (…){

        __shared__ int shared_array[512]; // static shared memory
… }

static_shered_mem<<<BLOCK_SIZE, GRID_SIZE>>> (…);
```

Dynamic shared memory - known at runtime – size passed to the kernel

```
__global__ dynamic_shered_mem(args){

        extern __shared__ int shared_array[]; // dynamic shared memory
… }

dynamic_shered_mem<<<BLOCK_SIZE, GRID_SIZE, SHARED_MEM_SIZE>>> (…);
```

Shared memory example

- Two types of particles (A and B) in a 1D periodic box → circle
- Force depends on the coordinate difference $f(x_i, x_j) \sim (x_i - x_j)$
- Only particles on the same circle interact
- Simulation setup:
  - Number of circles = number of blocks
  - Number particles in the box = number of threads per block
- We want to calculate force only between particles of type A
- We have variable number of particles (set by **spacer** variable) of type B between particles A
- Coordinates of all particles are stored in a linear array by their particle ID

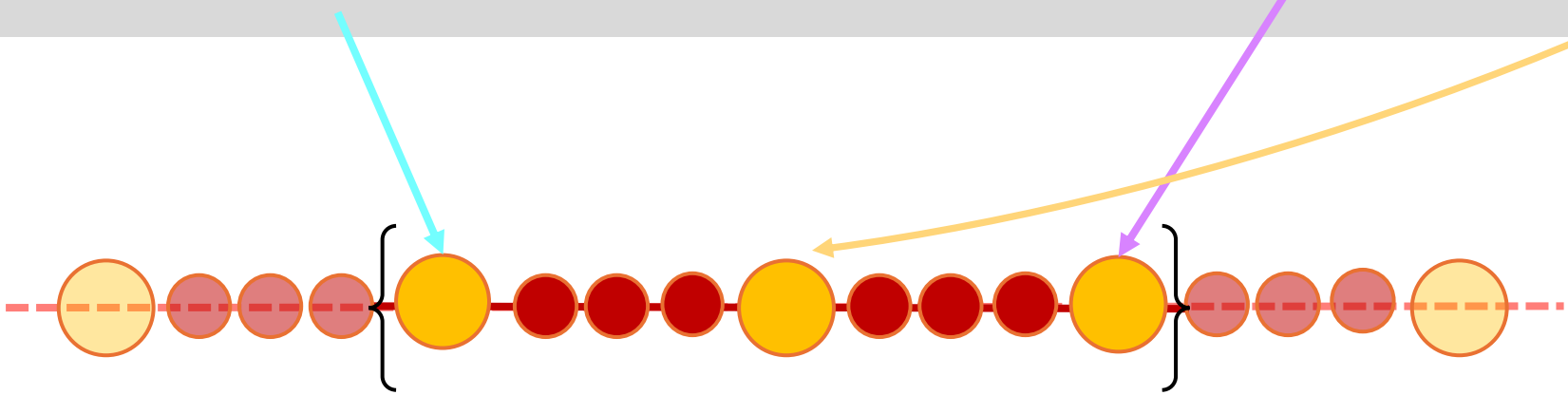


```
ssh your_user_name@frontera.tacc.utexas.edu
idev –p rtx
module load cuda
cd 4_SharedMemory
nvcc –o forceGPU forceGPU.cu
./forceGPU
```

```
=== Performance Statistics ===
Spacer:1
CPU time: 7.536716 ms
GPU time: 0.009787 ms
GPU time with shared memory: 0.006758 ms
```

```
=== Performance Statistics ===
Spacer:32
CPU time: 8.187716 ms
GPU time: 0.104289 ms
GPU time with shared memory: 0.049056 ms
```

```
__global__ void ForceGPU(float *d_force, float *d_pos, int N, int spacer){

int tid = threadIdx.x;
int offset = blockDim.x * blockIdx.x;

float my_pos = d_pos[spacer*(tid + offset)];

d_force[tid + offset] = (d_pos[spacer*((tid+1)%N + offset)] - my_pos) + (d_pos[spacer*((N + tid-1)%N + offset)] -
my_pos);}
```



```
__global__ void ForceGPUShared(float *d_force, float *d_pos, int N, int spacer){


extern __shared__ float shared_pos[];


int tid = threadIdx.x;

int gid = blockDim.x * blockIdx.x + threadIdx.x;


shared_pos[tid] = d_pos[gid * spacer]; //each threads writes its position to shared memory

float my_pos = d_pos[gid * spacer];

__syncthreads();

d_force[gid] = (shared_pos[(tid+1)%N] - my_pos) + (shared_pos[(N + tid-1)%N] - my_pos);}
```

```
ForceGPUShared<<<gridSize, blockSize, shared_mem_size >>>(…);
```

To interact with *m* nearest neighbors instead:

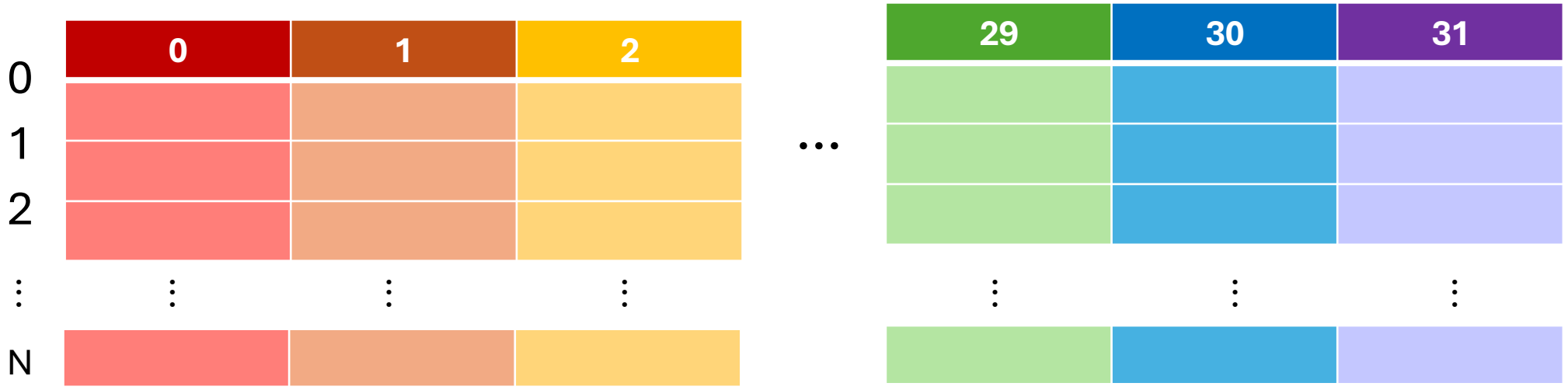
```
for(int m = 0; m < 3; m++)

force[i] += (pos[spacer*((inx+m)%len + offset)] - my_pos) +
                        (pos[spacer*((len+inx-m)%len + offset)] - my_pos);
```

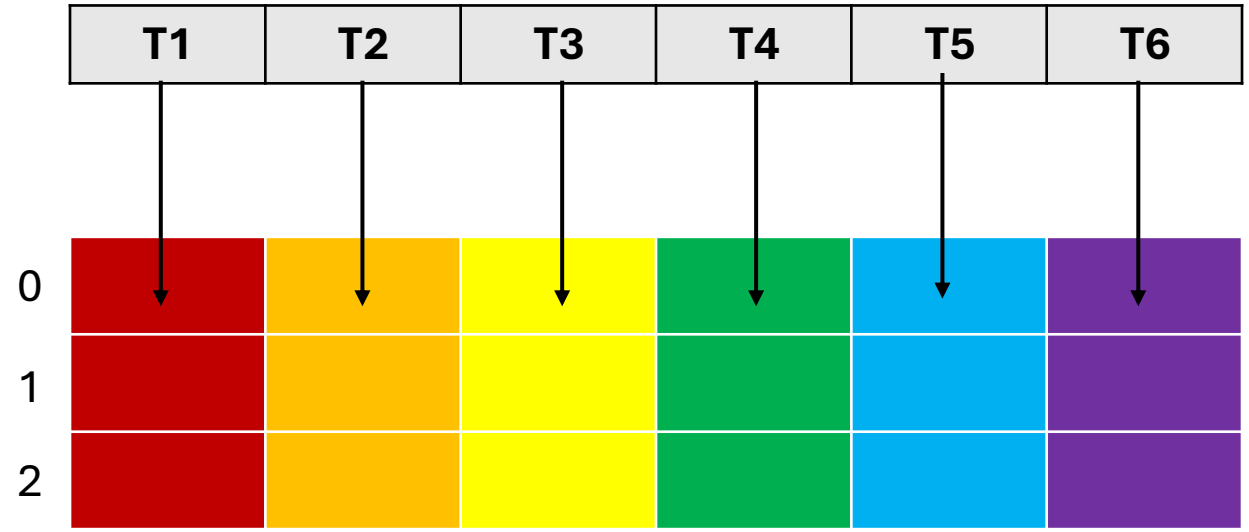
CPU timing

```
#include <chrono>

auto start_cpu = std::chrono::high_resolution_clock::now();

        ForceCPU(args);

auto end_cpu = std::chrono::high_resolution_clock::now();

cpu_time= std::chrono::duration_cast<std::chrono::nanoseconds>(end_cpu - start_cpu).count();
```

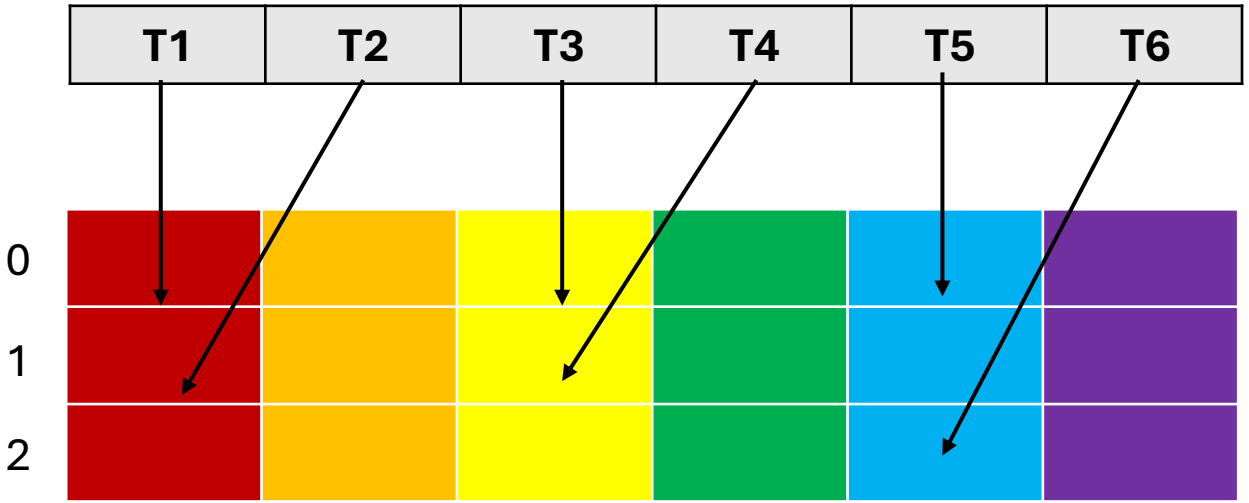
Shared memory and bank conflicts

- Shared memory arranged in 32 banks, 4 bytes each.
- One row (128 bytes) are accessed per memory transaction
- **Only one** row can be accessed at once. If two threads request data in the same bank, we have a bank conflict, two operations needed.





No bank conflicts, all data can be loaded in one transaction

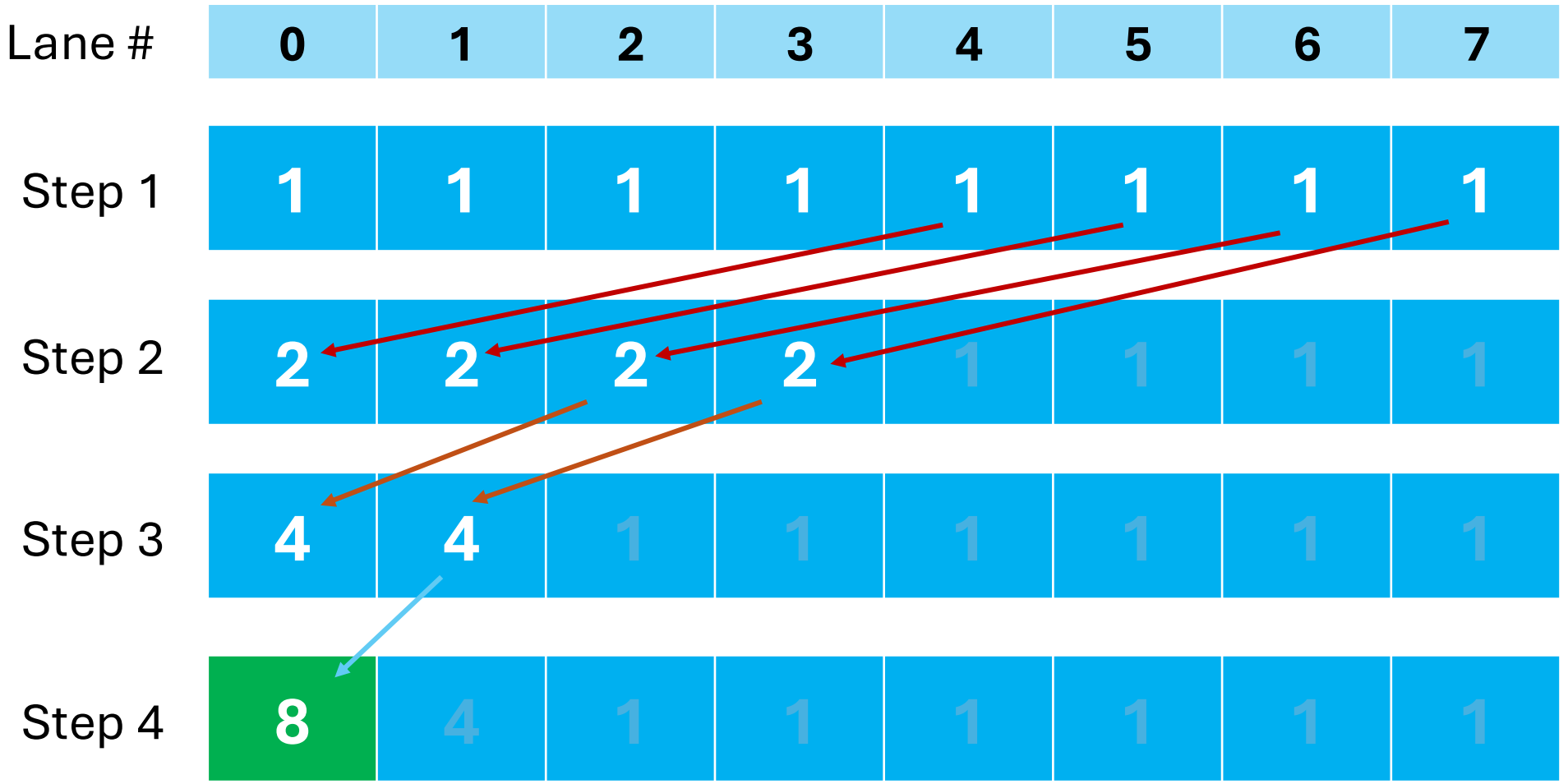


Separate transactions needed for all rows

Warp-level primitives

- Performance can be further improved by using warp-level collective operations and communication such as reductions, shuffles, scans, etc.
- Warp-level primitives enable data to be directly exchanged between thread registers instead of using shared memory (faster).
- Primitives use a 32-bit mask argument to select participating threads.
  Reminder: each thread in the warp will be assigned to one of the 0-31 lanes using threadIdx.x% warp_size
- Threads within the warp can be synchronized with *__syncwarp(mask)*.

Parallel reduction



```
ssh your_user_name@frontera.tacc.utexas.edu
idev –p rtx
module load cuda

cd 5_WarpFunctions
nvcc –o warp_primitives warp_primitives.cu
./warp_primitives
```

Important functions

```
__shfl_down_sync(mask, val, delta);
```

- Calling thread receives the value of variable val from the thread at
- laneID = own_laneId + delta

```
unsigned mask = __ballot_sync(unsigned mask, int predicate);
```

Returns a mask of threads for which the predicate evaluated to true

```
unsigned mask = __activemask();
```

Returns a mask of currently active threads in a warp

- Core loop of the kernel: each thread adds the value from a lane with a higher id (own_laneId + offset) to its own value. Offset gets halved until it reaches zero and final sum is at lane 0.
- Example below uses mask in which all threads are active.

```
#define FULL_MASK 0xffffffff
val = x[threadIdx.x];
for (int offset = 16; offset > 0; offset /= 2){
            val += __shfl_down_sync(FULL_MASK, val, offset);
```

To restrict the sum to the first M elements

```
unsigned mask = __ballot_sync(FULL_MASK, threadIdx.x%32 < M);

if (threadIdx.x%32 < M){
   int val = x[threadIdx.x];
   for (int offset = 16; offset > 0; offset /= 2){
      val += __shfl_down_sync(mask, val, offset);
```